13.2.2 Joins

Suppose you want to produce a list of all `Corvettes` in the database that have CD players. To do this, you need information from two tables, `Corvettes` and `Equipment`. The connection between these two tables is through the cross-reference table `Corvettes_Equipment`. The `SELECT` command allows the temporary construction of a virtual table that includes information from the `Corvettes` and `Equipment` tables, using the `Corvettes_Equipment` table as the basis for producing the desired result. Such a virtual table is built with a *join* of the two tables. A join is specified with a `SELECT` command that has two tables named in the `FROM` clause and uses a compound `WHERE` clause. The `WHERE` clause for our example must have three conditions. First, the `Vette_id` column from the `Corvettes` table must match the `Vette_id` column from the `Corvettes_Equipment` table. This restricts the rows of the `Corvettes_Equipment` table to those associated with the row of interest in the `Corvettes` table. Second, the `Equip` column from the `Corvettes_Equipment` table must match the `Equip_id` column of the `Equipment` table. This restricts the rows of the `Equipment` table to those associated with the row of interest of the `Corvettes_Equipment` table. Finally, the `Equip` column from the `Equipment` table must be `CD`. The complete `SELECT` command to extract the cars with CD players follows:

```
SELECT Corvettes.Vette_id, Corvettes.Body_style,
       Corvettes.Miles, Corvettes.Year, Corvettes.State,
       Equipment.Equip
FROM Corvettes, Equipment
WHERE Corvettes.Vette_id = Corvettes_Equipment.Vette_id
   AND Corvettes_Equipment.Equip = Equipment.Equip_id
   AND Equipment.Equip = 'CD';
```

This query produces the following result:

VETTE_ID	BODY_STYLE	MILES	YEAR	STATE	EQUIPMENT
1	coupe	18.0	1997	4	CD
2	hatchback	58.0	1996	7	CD
8	convertible	17.0	1999	5	CD
9	hardtop	17.0	2000	5	CD
10	hatchback	50.0	1995	7	CD

Notice that all references to columns in this query are prefixed with the table names. This is necessary only when the column names are not unique to one table, as is the case for the `Vette_id` column, which appears in both the `Corvettes` and the `Corvettes_Equipment` tables. However, even if the column names are unique, including the table names makes the query more readable.

As another example of a join, notice that the `State` column of the `Corvettes` table does not store state names. Instead, it stores row references to the `States` table, which stores state names. Any user who submits a query on

the Corvettes table would likely prefer that the states' names be returned rather than the reference to the States table. This can be easily accommodated in SQL. For example, suppose you want to get a list of the Corvettes for sale in California. This could be obtained with the following command:

```
SELECT Vette_id, Body_style, Year, States.State
FROM Corvettes, States
WHERE Corvettes.State = States.State_id AND
      States.State = 'California';
```

This query produces the following result:

```
VETTE_ID    BODY_STYLE    MILES     YEAR     STATE
    5       hatchback     25.0      1991     California
    8       convertible   17.0      1999     California
    9       hardtop       17.0      2000     California
```

13.2.3 The INSERT SQL Command

The INSERT command is used to add a row of data to a table. Its general form is as follows:

```
INSERT INTO table_name(column_name_1, column_name_2, ...,
     column_name_n)
VALUES (value_1, value_2, ..., value_n);
```

The correspondence between the column names and the values is positional: The first value goes into the column that is named first, and so forth. If INSERT is used on a table that has a column with the constraint NOT NULL and that column is not named in the INSERT, an error will be detected and reported. As an example of an INSERT command, consider the following:

```
INSERT INTO Corvettes(Vette_id, Body_style, Miles, Year,
                      State)
VALUES (37, 'convertible', 25.5, 1986, 17);
```

13.2.4 The UPDATE SQL Command

The UPDATE command is used to change one or more of the values of a row of a table. Its general form follows:

```
UPDATE table_name
SET column_name_1 = value_1,
    column_name_2 = value_2,
    ...
    column_name_n = value_n
WHERE column_name = value;
```

The WHERE clause in an UPDATE command specifies the primary key of the row to be updated. For example, to correct an error, you could change the year of the

row with `Vette_id = 17` in the `Corvettes` table to `1996` with the following command:

```
UPDATE Corvettes
SET Year = 1996
WHERE Vette_id = 17;
```

13.2.5 The DELETE SQL Command

One or more rows of a table can be deleted with the `DELETE` command, whose general form is as follows:

```
DELETE FROM table_name
WHERE column_name = value;
```

The `WHERE` clause specifies the primary key of the row to be deleted. For example, if the car with the `Vette_id` value `27` is sold and should no longer be in the database, you could remove it from the `Corvettes` table with the following command:

```
DELETE FROM Corvettes
WHERE Vette_id = 27;
```

The `WHERE` clause of a `DELETE` command can specify more than one row of the table, in which case all rows that satisfy the `WHERE` clause are deleted.

13.2.6 The DROP SQL Command

The `DROP` command can be used to delete either whole databases or complete tables. The general form is as follows:

```
DROP (TABLE | DATABASE) [IF EXISTS] name;
```

In this line, the parentheses and brackets are metasymbols. `DROP` is used with either `TABLE` or `DATABASE`. The `IF EXISTS` clause is included if you want to avoid errors if the named table or database may not exist. For example:

```
DROP TABLE IF EXISTS States;
```

13.2.7 The CREATE TABLE SQL Command

A table in a database can be created with the `CREATE` command, whose general form is as follows:

```
CREATE TABLE table_name(
    column_name_1     data_type  constraints,
    column_name_2     data_type  constraints,
    ...
    column_name_n     data_type  constraints);
```

A large number of different data types exist for table data, including INTEGER, REAL, DOUBLE, and CHAR(*length*).[4] There are also several different constraints, which can be somewhat different among various database vendors. Constraints are restrictions on the values that can appear in a column of a table. One common constraint is NOT NULL, which means that every row in the table must have a value in a column that has this constraint. Another common one is PRIMARY KEY, which means the column that has this constraint has a unique value for each row in the table. For example, you could have the following:

```
CREATE TABLE States(
   State_id INTEGER PRIMARY KEY NOT NULL,
   State CHAR(20));
```

In some situations, table columns are referenced by position number rather than by names. The columns of a table are numbered starting with 1; that is, the first column is column 1.

We have now introduced enough SQL to make the topics in the remainder of this chapter understandable.

13.3 Architectures for Database Access

A database can be made available to its users in several ways. The most common of these are briefly introduced in the following sections.

13.3.1 Client/Server Architectures

The basic client/server architecture of the Web was discussed earlier in this book. In any client/server configuration, part of the work is done by the client, and part is done by the server. A client/server database access architecture is very similar. The client machines provide a way for users to input requests to a database that is resident on a computer that runs a database server. Results of requests to the server are returned to the client, which may use them in subsequent computations or simply display them for the user. A database server implements a data manipulation language that presents an interface to clients. This language can directly access and update the database. In its simplest form, a client/server database configuration has only two components, the client and the server. Such systems are called *two-tier* systems.

In some cases, two-tier systems are adequate. For example, in simple uses of the Web, the server provides HTML documents, and the client displays them. There is little computation to be divided between the two. However, some other applications require a great deal more complexity than the Web. In recent years, large database servers have been replaced by multiple smaller servers, thus lessening the capabilities of the individual servers to deal with increasing

4. More SQL data types and their corresponding Java data types are shown in Table 13.1 in Section 13.7.

application complexity. At the same time, client systems have grown in power and sophistication. It would seem natural for the computational load in client/server systems to gravitate toward the clients. Unfortunately, there are other problems with this solution—specifically, if any part of the application is moved to the clients, there is the problem of keeping the clients current with changes in the applications that use the database. This is clearly a serious problem if there are a large number of clients.

The most common solution to the problems of two-tier systems is to add a third component, thereby hatching a three-tier architecture. The first tier has the Web browser, which provides the user interface. The middle tier of such a system usually has the Web server and the applications that require database access. The third tier in the system has the database server and the database itself. The architecture of a three-tier Web-based database access system has the form shown in Figure 13.6.

Figure 13.6 Three-tier architecture of a Web site supported by databases

13.3.2 The Microsoft Access Architecture

Microsoft Access is a tool for implementing database applications that can access databases from virtually any common database vendor. It provides access to different database systems in two different ways: through its Jet database engine or through the Open Database Connectivity (ODBC) standard. ODBC specifies an application programming interface (API) for a set of objects and methods that serves as an interface to different databases. Each database must have a driver, which is an implementation of these objects and methods. Vendors for most common databases provide ODBC drivers. By using ODBC, an application can include SQL statements (through the ODBC API) that work with any database for which a driver has been installed. A system called the *ODBC driver manager*, which runs on the client computer, chooses the proper driver for a request on a specific database.

13.3.3 The Perl DBI/DBD Architecture

Perl programs use a two-stage system to access databases. First, the database interface (DBI) module provides a collection of methods and attributes that allows generic SQL access to databases. The second stage is the direct database interface, called a database driver (DBD), which is specific to the particular rela-

tional database being used. There are DBDs for most common relational databases. Programmers use the Perl DBI to write applications, which are nearly independent of the particular database to be used. The Perl DBI module is discussed in Section 13.5.

Perl CGI programs provide Web access to databases, using the DBI and the DBDs.

13.3.4 PHP and Database Access

PHP includes support for a wide variety of database systems. For each supported database system, there is an associated API. These APIs provide the interface to the specific systems. For example, the MySQL API includes functions to connect to a database and apply SQL commands against the database. Web access to a database using PHP is a natural architecture because PHP scripts are called through HTML documents from browsers. Using PHP and MySQL for database access is discussed in Section 13.6.

13.3.5 The Java JDBC Architecture

The Java JDBC architecture is a Java API for database access.[5] JDBC is very similar to ODBC, at least in terms of purpose. Both have the X/OPEN SQL Call Level Interface (SQL CLI) in their heritages.

JDBC provides a standard set of interfaces between applications that use databases and the low-level access software that actually manipulates the databases, which is supplied by the database vendor and is dependent on the particular brand of database being used. JDBC allows applications to be independent of the database system being used, as long as a JDBC driver is installed on the platform on which the application is run.

The advantages of JDBC are basically those of Java: The language is expressive and relatively safe, and programs are highly portable among platforms. The disadvantage of JDBC is that Java/JDBC programs are more complex than programs that accomplish the same things but are written in Perl or PHP.

JDBC is described in Section 13.7.

Figure 13.7 shows the most common database access architecture. Microsoft's Access architecture uses ODBC for its database API. For Perl, the API is DBI. PHP uses collections of functions as APIs for different databases. Java has JDBC as its database API.

5. JDBC sounds like an acronym for Java Database Connectivity, but Sun Microsystems has denied this. In fact, Sun has registered JDBC as a trademark but has not done the same for Java Database Connectivity.

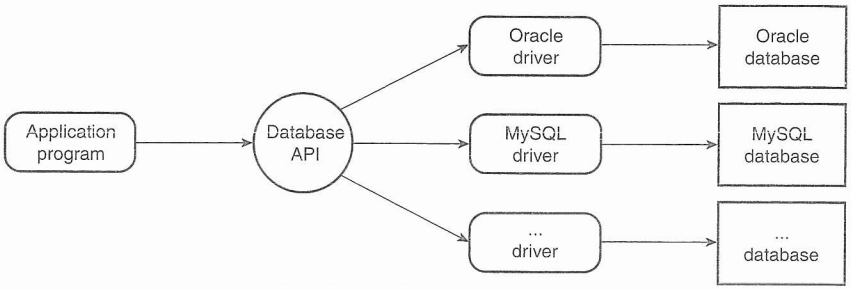


Figure 13.7 Common database access architecture

13.4 The MySQL Database System

MySQL is a free, efficient, widely used database system that implements SQL. It is available for all widely used computing platforms. MySQL software and documentation can be downloaded from `http://www.mysql.org`. Some Linux system distributions, such as the one from Red Hat, include MySQL. This section describes a small part of MySQL. As with other software systems illustrated in this book, we do not discuss how to install or manage MySQL. These are usually system administration tasks. We only cover the use of MySQL, not its administration.

The first step when using MySQL is logging in to the MySQL system, which is done with the following command (at the command line of the operating system):

`mysql [-h` *host*`] [-u` *username*`] [`*database_name*`] [-p]`

The parts of this command that are in square brackets are optional. The *host* is the name of the server running MySQL; if absent, MySQL assumes it is the user's machine. If *username* is absent, it assumes that the name you used to log on to the machine is the correct username. If *database_name* is given, that database is selected as the focus of MySQL, making it the object of subsequent commands. If `–p` is included, it means a password is required, for which MySQL will ask.

Once you have successfully logged into MySQL, it is ready to receive commands. Although it is called "logging on," what you are actually doing is starting execution of the MySQL system.

If the database to be accessed already exists but its name was not included when logging in to MySQL, the `use` command can be used to focus on the database of interest. For example, if we want to access a database named `cars`, we could use the following:

```
use cars;
```

This is sometimes called making a specific database the "current" database for the MySQL server. The MySQL response to this command is as follows:

```
Database changed
```

This seems odd because no change has been made to a database. Note the semicolon at the end of the `use` command—it is essential here, as it is for all MySQL commands. If a command is given without a semicolon, MySQL will wait indefinitely for one. Until a semicolon is found, MySQL behaves as if the remainder of the command will follow.

If a database is not specified when logging in to MySQL and a database command is given before `use` is used to focus on a database, the following error message will be issued:

```
ERROR 1046: No Database Selected
```

If a new database is to be created, the database itself must be created first and then the tables that will make up the database. A new database is created with the SQL `CREATE DATABASE` command. For example:

```
CREATE DATABASE cars;
```

This command also elicits an odd response from MySQL:

```
Query ok, 1 row affected (0.05 sec)
```

The time given varies with the speed of and load on the host machine.

The tables of a database are created with the `CREATE TABLE` command, whose syntax is that of SQL. For example:

```
CREATE TABLE Equipment
   (Equip_id  INT  UNSIGNED  NOT NULL  AUTO_INCREMENT
              PRIMARY KEY,
    Equip  CHAR(10)
   );
```

The `INT` and `UNSIGNED` parts of the `Equip_id` column indicate the data type. The `AUTO_INCREMENT` is a MySQL convenience. It specifies that the values of this column need not be given when populating the table. The values `1`, `2`, `3`, and so forth will be implicitly assigned. The value `NULL` is given in place of a value for a column so specified when populating the table with `INSERT`. A large number of different data types are possible for field values. The most common of these are `CHAR(`*length*`)`, `INT`, and `FLOAT(`*total*,*fractional*`)`, where *total* specifies the total number of characters, including both digits and the decimal point, and *fractional* gives the number of digits to the right of the decimal point.

The `SHOW` command can be used to display the tables of the database, as in the following:

```
SHOW TABLES;
```

If our sample database, `cars`, is the database of current focus, this produces the following:

```
--------------
show
--------------
```

```
+---------------------+
|                     |
| Tables_in_cars      |
|                     |
+---------------------+
| Corvettes           |
| Corvettes_Equipment |
| Equipment           |
| States              |
+---------------------+
```

The `DESCRIBE` command can be used to display the description of the structure of a table. For example,

```
DESCRIBE Corvettes;
```

produces the following:

```
+----------+----------------+-----+----+--------+--------------+
|Field     |Type            |Null |Key |Default |Extra         |
+----------+----------------+-----+----+--------+--------------+
|Vette_id  |int(10) unsigned|     |PRI |NULL    |auto_increment|
+----------+----------------+-----+----+--------+--------------+
|Body_style|char(12)        |     |    |        |              |
+----------+----------------+-----+----+--------+--------------+
|Miles     |float(4,1)      |     |    |0.0     |              |
+----------+----------------+-----+----+--------+--------------+
|Year      |int(10) unsigned|     |    |0       |              |
+----------+----------------+-----+----+--------+--------------+
|State     |int(10) unsigned|     |    |0       |              |
+----------+----------------+-----+----+--------+--------------+
```

The other MySQL commands that are needed here—`INSERT`, `SELECT`, `DROP`, `UPDATE`, and `DELETE`—are all implementations of their corresponding SQL commands. Therefore, their descriptions do not need to be repeated in this section.

There are many tools available to aid in database administration (for example, from `http://dev.mysql.com`). For example, MySQL Administrator is a program that performs configuring, monitoring, starting and stopping a MySQL server, managing users and connections, performing backups, and several other administrative tasks.

13.5 Database Access with Perl and MySQL

Access to a database from a Perl program is a two-stage process. A standard Perl object-oriented module, DBI (for *da*ta*b*ase *i*nterface), provides an API for database access. An application program uses the methods and attributes of DBI to specify database accesses. The second stage of database access from Perl is a database driver (DBD) module. Each different database system requires its own driver, which is the interface to the actual database. So, a Perl program uses DBI to access a specific DBD, which accesses the physical database. This approach allows a Perl database application program to be relatively independent of the particular database it will access.

13.5.1 The DBI Module

The DBI module provides a large number of different methods and attributes, only a few of which are discussed in this section. A more complete description of DBI can be obtained by typing `perldoc DBI` at the operating system command.

The user interface design of DBI is related to that for files in Perl. Database operations are referenced through a database handle, which is defined by an operation that opens the database for access. DBI is used as an object-oriented module, so its methods and attributes are referenced through a reference variable. The first step in database access is to open the database and set a database handle variable to reference the object that represents the database in the application program. Before any DBI operations can be done, the DBI module must be made available with a `use` statement:

```
use DBI;
```

This statement creates a `DBI` object, which can be referenced through the variable `DBI`.

A database is opened with the `DBI` class method `connect`, which takes three parameters, although the last two are optional. Its protocol is as follows:

```
$dbh = DBI->connect("DBI:driver_name:database_name"
                    [, username][, password]);
```

The first parameter specifies the DBI module name, the database driver name, and the name of the database to which the connection is to be made. The driver name for MySQL is `mysql`. If the username is not specified, the login name is used; if the database does not require a password, one does not need to be included. For example, we could have the following:

```
$dbh = DBI->connect("DBI:mysql:cars");
```

This creates a database handle for the `cars` database and stores it in `$dbh`. It assumes the username of the user who logged in is to be used for database access. Finally, it assumes the database does not need a password.

Because the connect operation can fail (for example, if the specified database does not exist), the call to `connect` is usually called in conjunction with a `die` operator. When `connect` fails, it returns an object that evaluates to `false`. In this case, `connect` also produces an error message that is returned from the `errstr` method of the handle it returns. Therefore, the following is a typical call to `connect`:

```
$dbh = DBI->connect("DBI:mysql:cars") or
   die "Error -- unable to open cars: $dbh->errstr\n";
```

In the `die` expression, `$dbh->errstr` refers to an error message.

The database handle is actually an object that encapsulates the DBD for the specific database to be accessed. A Perl program can have connections to any number of different databases, each of which could be from a different vendor. Once the database is open and the connection to it has been made, it can be accessed. Although the handle method `do` can be used to request the execution of an SQL command, it is sometimes more convenient first to create a compiled version of an SQL command with the method `prepare`, which returns an object reference to the command. This object is called a *statement handle*. Once the statement handle has been created, it can be executed with the statement method `execute`. For example, the second query from Section 13.2.2 could be compiled with the following:

```
$sth = $dbh->prepare("SELECT Vette_id, Body_style, Year,
                             States.State
                      FROM Corvettes, States
                      WHERE Corvettes.State = States.State_id
                             AND
                            States.State = 'California'");
```

Because `execute` can fail, the call to it is usually combined with a call to `die`. For example:

```
$sth->execute() or
   die "Error -- unable to execute query: $dbh->errstr \n";
```

The `$sth` statement handle object now has the results of the executed query.

In general, to display the results of database queries properly, the names of the result columns are needed. The statement handle object has an attribute, `NAME`, which is a reference to an array that has the names of the columns in the result. The attributes are stored in a hash, so `NAME` is used as a hash subscript to get the array of column names, as in the following:

```
$col_names = $sth->{NAME};
```

The rows of the returned value can be gotten with the `fetchrow_array` method of the statement handle object. This method returns a reference to an array with the next row of the result. If there are no more rows in the result, `fetchrow_array` returns `false`.

When the program is finished with a particular statement handle, it should call the `finish` method on the object. At the end of the program, the `disconnect` method of the database handle should be called.

13.5.2 An Example

When a query is made on a database through a browser, the result of the query must be returned to the browser as HTML. Putting database field data into an HTML document creates a potential problem. A field retrieved from the database may contain characters that are special in HTML, namely >, <, ", or &. These can be converted to their corresponding HTML entities with the `CGI.pm` function `escapeHTML`, which is used in the following example.

The following is a complete CGI Perl program, `access_cars.pl`, which takes a query from a text area widget of a form in an HTML document. The program connects to the `cars` database, performs the query, and creates an HTML table of the results.

```
#!/usr/bin/perl -w
# access_cars.pl
# A CGI program to illustrate using MySQL from Perl

# Get access to DBI and CGI

use DBI;
use CGI ":standard";

print header();
print start_html("CGI-Perl MySQL database access");

#>>> Create the connection to the database, cars
my $dbh = DBI->connect("DBI:mysql:cars", "root", "");
if (!$dbh) {
    print "Error connecting to database; $dbh->errstr\n";
}

#>>> Get the query and display it
my $query = param("query");
print "<p> <b> The query is: </b>", $query, "</p>";

#>>> Build a statement object for a SELECT SQL command
my $sth = $dbh->prepare($query);

#>>> Execute the statement
$sth->execute or
```

```
    die "Error - unable to execute query: $dbh->errstr\n";

#>>> Get a reference to the column names in the returned value and
#>>>  display the column names as the first table row
print "<table> <caption> <h2> Query Results </h2> </caption>",
      "<tr align = 'center'>";
my $col_names = $sth->{NAME};
foreach $field_name (@$col_names) {
    print "<th> $field_name </th>";
}
print "</tr>";

#>>> Get the rows of the result and display them in the table
while (@result_rows = $sth->fetchrow_array) {
    print "<tr align = 'center'>";
    while ($#result_rows >= 0) {
        $field = shift @result_rows;

#>>> Replace the HTML special characters with their entities
        $field = escapeHTML($field);
        print "<td>  $field </td>";
    }
    print "</tr>";
}
print "</table>";
$sth->finish;
$dbh->disconnect;
print end_html();
```

Figure 13.8 shows a browser display of the output of `access_cars.pl`.

The query is: SELECT * FROM Corvettes, States WHERE Corvettes.State = States.State_ AND States.State = "California"

Query Results

Vette_id	Body_style	Miles	Year	State	State_id	State
5	hatchback	25.0	1991	5	5	California
8	convertible	17.0	1999	5	5	California
9	hardtop	17.0	2000	5	5	California

Figure 13.8 Display of the output of `access_cars.pl`

There is, of course, much more to DBI than we have given here. However, there is sufficient information to introduce the reader to it, which is our goal.

13.6 Database Access with PHP and MySQL

PHP access to a database is often done with two HTML documents: one to collect a user request for a database access and one to host the PHP code to process the request and generate the return HTML document. The user request collector is a simple HTML document. Therefore, this section is primarily about the database connection and processing.

13.6.1 Potential Problems with Special Characters

In Section 13.5.2, we explained the potential problem with characters in a database that are special in HTML (>, <, ", and &). PHP includes a function, `htmlspecialchars`, that replaces all occurrences of these four special characters in its parameter with their corresponding entities. For example, consider the following code:

```
$str = "Apples & grapes <raisins, too>";
$str = htmlspecialchars($str);
```

After the interpretation of this code, the value of `$str` has the following value:

```
"Apples &amp; grapes &lt;raisins, too&gt;"
```

This string is now ready to be made the content of an HTML tag without causing any browser confusion.

Another problem with special characters can occur with PHP scripts that get values through `GET`, `POST`, or from a cookie. Strings from these sources could include single quotes, double quotes, backslashes, and null characters, all of which could possibly cause problems if they are used in other strings in a script. To avoid these problems, the PHP system has an implicit backslashing function named `magic_quotes_gpc`, which can be turned on or off in the `PHP.ini` file. When this function is enabled, which is the default, all values received in a script from `$_POST`, `$_GET`, and `$_COOKIE` have backslashes implicitly inserted in front of all single quotes, double quotes, backslashes, and null characters. This avoids any problems that could be caused by those characters. For example, if the string `O'Reilly` is fetched from `$_POST`, it would be converted by `magic_quotes_gpc` to `O\'Reilly`. Unfortunately, this causes other problems. If the script compares the name to a nonslashed version, the comparison will fail. Furthermore, even displaying the name will show the backslash.

This problem is relevant here because we want to have a PHP script get SQL commands from a text box in an XHTML document. For example, suppose `magic_quotes_gpc` is on and the value for a query obtained from a text box on a form is as follows:

```
SELECT * FROM Corvettes WHERE Body_style = 'coupe'
```

If the name of the text box is `query`, its value is put in `$query` with the following statement:

```
$query = $_POST['query'];
```

The value of `$query` is converted to the following by `magic_quotes_gpc`:

```
SELECT * FROM Corvettes WHERE Body_style = \'coupe\'
```

Unfortunately, this string is not a legal SQL command (because of the backslashes). If it is sent to MySQL as a command, MySQL will reject it and report an error. Therefore, if complete SQL commands are to be collected from a form, as is done with the Perl/CGI program `access_cars.pl`, `magic_quotes_gpc` must be disabled in `PHP.ini` to avoid the extra backslashes. The alternative to changing the value of `magic_quotes_gpc` is to remove the extra slashes in the PHP script with the predefined function `stripslashes`, as in the following:

```
$query = stripslashes($query);
```

13.6.2 Connecting to MySQL and Selecting a Database

The PHP function `mysql_connect` connects a script to a MySQL server. This function takes three parameters, all of which are optional. The first is the host that is running MySQL; the default is localhost (the machine on which the script is running). The second parameter is the username for MySQL; the default is the username in which the PHP process runs. The third parameter is the password for the database; the default is blank (works if the database does not require a password). For example, if the default parameters were acceptable, we could use the following:

```
$db = mysql_connect();
```

Of course, the connect operation could fail, in which case the value returned would be `false` (rather than a reference to the database). Therefore, the call to `mysql_connect` usually is used in conjunction with `die`.

The connection to a database is terminated with the `mysql_close` function. This function is not necessary when using MySQL through a PHP script because the connection will be closed implicitly when the script terminates.

When running MySQL from the command line, a database must be selected as the current, or focused, database. This is also necessary when using MySQL through PHP; it is accomplished with the `mysql_select_db` function, as shown in the following:

```
mysql_select_db("cars");
```

13.6.3 Requesting MySQL Operations

MySQL operations are requested through the `mysql_query` function. Typically, the operation, in the form of a string literal, is assigned to a variable. Then `mysql_query` is called with the variable as its parameter. For example:

```
$query = "SELECT * from Corvettes";
$result = mysql_query($query);
```

The return value from `mysql_query` is used to identify, internally, the data that resulted from the operation. In most cases, the first thing to do with the result is to determine the number of rows. This is obtained with the `mysql_num_rows` function, which is given the result value returned by `mysql_query`, as shown in the following:

```
$num_rows = mysql_num_rows($result);
```

The number of fields in a result row can be determined with `mysql_num_fields`, as shown in the following:

```
$num_fields = mysql_num_fields($result);
```

The rows of the result can be retrieved into several different forms. We will use `mysql_fetch_array`, which returns an array of the next row. Then the field values can be obtained by subscripting the return array with the column names. For example, if the result of a query had columns for `State_id` and `State`, we could display the results with the following code:

```
$num_rows = mysql_num_rows($result);

for ($row_num = 1; $row_num <= $num_rows; $row_num++) {
   $row = mysql_fetch_array($result);
   print "<p> Result row number" . $row_num .
         ". State_id: ";
   print htmlspecialchars($row["State_id"]);
   print " State: ";
   print htmlspecialchars($row["State"]);
   print "</p>";
}
```

The situation in which the column names are not known is considered in Section 13.6.4, which includes a complete example of accessing a database through PHP and MySQL.

13.6.4 A PHP/MySQL Example

One simple example of Web access to a database is to use an HTML form to collect a query from a user, apply the query to the database, and return a document that shows the results of the query. The form that allows users to input queries is simple. The PHP script to connect to the database and perform the query is also relatively simple—we have done all of this in Section 13.6.3. All that remains is to get the results into a form that is easy to present to the user. The example of displaying query results in Section 13.6.3 was easy because the names of the columns of the result were known and the results were not put in a table.

The rows of the result of a query are PHP arrays, which are also arrays. Such an array has double sets of elements, one with numeric keys and one with string keys. For example, if a query gets a row with the field values (1, Alabama), the row actually stores four hash elements, two with numeric keys and two with string keys. For the States table of the cars database, the result row would actually have the following:

```
((0, 1), (State_id, 1), (1, Alabama), (State, Alabama))
```

If a row is indexed with numbers, the element values are returned. For example, if a row of the result of a query is in $row, then $row[0] is the value of the first field in the row, $row[1] is the value of the second field, and so forth. The rows could be indexed with strings, in which case $row["State"] would have the value Alabama. As a result of this double storage of result fields, the result rows have twice as many elements as there are fields in the result. If only the values are needed, they can be fetched from the value part of every other hash element, beginning with the second (the element with subscript 1). The following will display all of the field values in the result row in $row:

```
$values = array_values($row);
for ($index = 0; $index < $num_fields; $index++)
    print "$values[2 * $index + 1] <br />";
```

When the results are being returned as HTML content, it is always a good idea to use htmlspecialchars on the field values.

Getting the column labels from the results of a MySQL query can be confusing. From the example of the actual contents of a result array previously shown, the column labels are the keys of the odd-numbered elements of the array (State_id and State). The keys can be displayed in the same way the values were displayed previously.

```
$keys = array_keys($row);
for ($index = 0; $index < $num_fields; $index++)
  print "$keys[2 * $index + 1] <br />";
```

The following is the HTML document carsdata.html to collect queries on the cars database from the user.

```
<!-- carsdata.html
     Uses a form to collect a query against the cars
     database.
     Calls the PHP script, access_cars.php to perform
     the given query and display the results
    -->
<html>
  <head><title> Access to the cars database </title>
  </head>
  <body>
```

```
    <p>
      Please enter your query:
      <br />
      <form action  = "access_cars.php" method = "post">
        <textarea  rows = "2"  cols = "80" name = "query" >
        </textarea>
        <br /><br />
        <input type = "reset"  value = "Reset" />
        <input type = "submit"  value = "Submit request" />
      </form>
    </p>
  </body>
</html>
```

The following HTML/PHP document, `access_cars.php`, processes a query and returns the results in a table.

```
<!-- access_cars.php
     A PHP script to access the cars database
     through MySQL
     -->
<html>
  <head>
    <title> Access the cars database with MySQL </title>
  </head>
  <body>
<?php

// Connect to MySQL
$db = mysql_connect("localhost", "rws", "");
if (!$db)
     exit("Error - Could not connect to MySQL");

// Select the cars database
$er = mysql_select_db("cars");
if (!$er)
    exit("Error - Could not select the cars database");

// Get the query and clean it up (delete leading and trailing
// whitespace and remove backslashes from magic_quotes_gpc)
$query = $_POST['query'];
trim($query);
$query = stripslashes($query);
```

```
// Display the query, after fixing html characters
$query_html = htmlspecialchars($query);
print "<p> <b> The query is: </b> " . $query_html . "</p>";

// Execute the query
$result = mysql_query($query);
if (!$result) {
    print "Error - the query could not be executed";
    $error = mysql_error();
    print "<p>" . $error . "</p>";
    exit;
}

// Display the results in a table
print "<table><caption> <h2> Query Results </h2> </caption>";
print "<tr align = 'center'>";

// Get the number of rows in the result, as well as the first row
//  and the number of fields in the rows
$num_rows = mysql_num_rows($result);
$row = mysql_fetch_array($result);
$num_fields = mysql_num_fields($result);

// Produce the column labels
$keys = array_keys($row);
for ($index = 0; $index < $num_fields; $index++)
    print "<th>" . $keys[2 * $index + 1] . "</th>";
print "</tr>";

// Output the values of the fields in the rows
for ($row_num = 0; $row_num < $num_rows; $row_num++) {
    print "<tr align = 'center'>";
    $values = array_values($row);
    for ($index = 0; $index < $num_fields; $index++) {
        $value = htmlspecialchars($values[2 * $index + 1]);
        print "<th>" . $value . "</th> ";
    }
    print "</tr>";
    $row = mysql_fetch_array($result);
}
print "</table>";
?>
  </body>
</html>
```

Figure 13.9 shows a browser display of the results of `access_cars.php` on the given query.

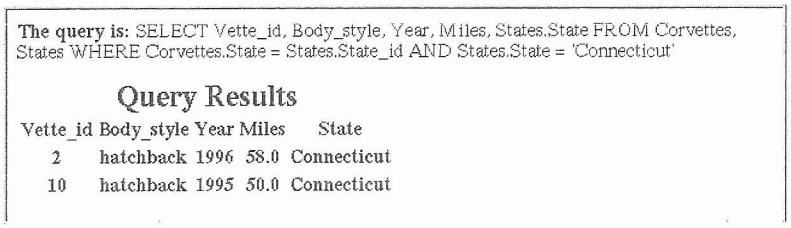


Figure 13.9 Display of the return document from `access_cars.php`

The two documents, `carsdata.html` and `access_cars.php`, which together collect a query from a user, apply it to the database, and return the results, can be combined. After inserting the XHTML markup from `carsdata.html` into `access_cars.php`, several modifications and additions must be made to the resulting document. First, the `action` attribute of the form must be changed to be self referential. One simple way to do this is to change the value to the name of the combined file. Next, there is the issue of how to get the PHP processor to produce the query collection markup the first time the document is requested and interpret the query processing code on the next request. The commonly used approach to this is to create a hidden input element that sets its value when the document is first displayed. The PHP code in the document checks the value of the hidden element to determine whether the action is to display a text area to collect a query or to apply the query to the database and display the result. The hidden element is defined with markup as shown here:

```
<input type = "hidden"  name = "stage"  value = "1" />
```

The PHP code to test the value of the hidden element has the following form:

```
$stage = $_POST["stage"];
if (!IsSet($stage)) { ... }
```

The then clause of this selector would contain the display of the form to collect the query. The else clause would contain the query processing and result display code. The combination of `carsdata.html` and `access_cars.php`, named `access_cars2.php`, follows.

```
<!-- access_cars2.php
     A PHP script to both get a query from the user and
     access the cars database through MySQL to get and
     display the result of the query.
     -->
```

```
<html>
  <head>
    <title> Access the cars database with MySQL </title>
  </head>
  <body>
<?php

// Is this the first request?
$stage = $_POST["stage"];
if (!IsSet($stage)) {
?>
    <p>
      Please enter your query:
      <br />
      <form  method = "POST"  action = "access_cars2.php" >
        <textarea  rows = "2"  cols = "80"  name = "query">
        </textarea>
        <br /><br />
        <input type = "hidden"  name = "stage"  value = "1" />
        <input type = "submit"  value = "Submit request" />
      </form>
    </p>
<?php
} else {  // $stage was set, so process the query

// Connect to MySQL
  $db = mysql_connect("localhost", "rws", "");
  if (!$db)
     exit("Error - Could not connect to MySQL");

// Select the cars database
  $er = mysql_select_db("cars");
  if (!$er)
    exit("Error - Could not select the cars database");

// Clean up the given query (delete leading and trailing
//  whitespace
  $query = $_POST['query'];
  trim($query);

// Fix the query for browser display and display it
  $query_html = htmlspecialchars($query);
  print "<p> <b> The query is: </b> " . $query_html . "</p>";
```

```
// Execute the query
  $result = mysql_query($query);
  if (!$result) {
    print "Error - the query could not be executed";
    $error = mysql_error();
    print "<p>" . $error . "</p>";
    exit;
  }

// Display the results in a table
  print "<table><caption> <h2> Query Results </h2> </caption>";
  print "<tr align = 'center'>";

// Get the number of rows in the result, as well as the first row
//  and the number of fields in the rows
  $num_rows = mysql_num_rows($result);
  $row = mysql_fetch_array($result);
  $num_fields = mysql_num_fields($result);

// Produce the column labels
  $keys = array_keys($row);
  for ($index = 0; $index < $num_fields; $index++)
    print "<th>" . $keys[2 * $index + 1] . "</th>";
  print "</tr>";

// Output the values of the fields in the rows
  for ($row_num = 0; $row_num < $num_rows; $row_num++) {
    print "<tr align = 'center'>";
    $values = array_values($row);
    for ($index = 0; $index < $num_fields; $index++){
      $value = htmlspecialchars($values[2 * $index + 1]);
      print "<th>" . $value . "</th> ";
    }
    print "</tr>";
    $row = mysql_fetch_array($result);
  }  // end of the for ($row_num = 0; ...
  print "</table>";
}  // end of the else clause
?>
</body>
</html>
```

13.7 Database Access with JDBC and MySQL

JDBC is a Java API for database access. A servlet can use JDBC to connect to a database and send SQL commands to the database as the parameter of a JDBC method. The Java interfaces that define JDBC are included in the `java.sql` package, which is part of the standard Java distribution.

13.7.1 JDBC and MySQL

This section describes the mechanisms for using JDBC to perform simple SQL operations on an existing database. The first step in developing a JDBC servlet is to establish a connection between the application and the database interface, or driver. The `DriverManager` class provides the method `getConnection`, which establishes the connection to the database. This class must select the correct driver for the database from those that have been registered.

The general form of a reference to a database for the connection operation is as follows:

`jdbc:`*subprotocol_name*`:`*more_info*

The *subprotocol_name* part is used to specify the driver for the database. For a MySQL database, the *subprotocol_name* is `mysql`. The *more_info* part is dependent on the subprotocol being used. If the database is local, it may be just the name of the database. If the database is somewhere else on the Internet, it may be the URL of the database. In the case of a database being on the same machine as the client, the *more_info* part includes `localhost` and the name of the database followed by a username and a password for the database. The username and password are attached to the database address in the same way HTML `GET` parameters are attached to a URL. For the sample MySQL database `cars`, assuming root is the user and there is no password, the reference is as follows:

```
jdbc:mysql://localhost/cars?user=root
```

A database driver may be registered in two ways, one general and the other specific. The general approach is to have the system property `jdbc.drivers` maintain a list of drivers. A servlet can add a new driver to this property by assigning the driver's class address to the property. For example, consider this statement:

```
jdbc.drivers = org.gjt.mm.mysql.Driver;
```

Here, `org.gjt.mm.mysql.Driver` is the name of the MySQL JDBC driver class that is used in the examples in this section. (The driver can be downloaded from `http://www.mysql.org`.) The only other requirement to make the driver usable is that it be loadable by the servlet, which means that it must be stored in a place where the servlet can access it. The driver manager is responsible for choosing the correct driver from those that are registered.

The less general way of registering a driver is to do it manually with the `forName` method of the `Class` class, giving the name of the driver class as a parameter:

```
Class.forName("org.gjt.mm.mysql.Driver").newInstance();
```

This approach is adequate if the servlet will be used exclusively for databases from one specific vendor.

The connection to a database from a servlet is made by creating a `Connection` object with the `getConnection` method of the `DriverManager` class. For the sample database `cars` and the MySQL database system, the `Connection` object can be created with the following statement:

```
myCon = DriverManager.getConnection(
              "jdbc:mysql://localhost/cars?user=root");
```

The `Connection` object is used to specify all database operations from the servlet.

After the appropriate database driver is registered and the connection to the database is established, a servlet can access the database using SQL commands. The first step in using SQL from a servlet is to create a `Statement` object through which one of the `Statement` methods can be used to actually issue the command. The `Statement` object is created with the `createStatement` method of the `Connection` class. If `myCon` is the `Connection` object, the following statement can be used:

```
Statement myStmt = myCon.createStatement();
```

SQL commands are created as `String` objects, as shown in the following example:

```
final String sql_com =
 "UPDATE Corvettes SET Year = 1991 WHERE Vette_id = 7";
```

From the JDBC point of view, there are two categories of SQL commands: the action commands, which include `INSERT`, `UPDATE`, `DELETE`, `CREATE TABLE`, and `DROP TABLE`; and the query command, `SELECT`. The action commands are executed through the `executeUpdate` method of the `Statement` object. For example, the previous SQL command `sql_com` can be executed with the following statement:

```
myStmt.executeUpdate(sql_com);
```

The `executeUpdate` method returns the number of rows that were affected by the command that it sent to the database.

A `SELECT` SQL command can be executed by sending it as the parameter to the `executeQuery` method of the `Statement` object. Executing a `SELECT` command differs from executing an action command in that the `SELECT` command is expected to return a part of the data found in the database. So, a call to `executeQuery` must be assigned to a program variable. The class of this vari-

able must be `ResultSet`, which is structured to store such results and which has methods to provide access to the data of the result. Consider the following example:

```
ResultSet result;
final String sql_com =
     "SELECT * FROM Corvettes WHERE Year <= 1990"
result = myStmt.executeQuery(sql_com);
```

Objects of the `ResultSet` class are similar to objects of classes that implement the related interface `Enumeration`. In both cases, the elements of the object are accessed through an iterator method. In the case of `Enumeration`, the iterator method is named `nextElement`; in the case of `ResultSet`, it is named `next`. The `next` method is a predicate—it returns a Boolean value, depending on whether there is another element in the `ResultSet` object. Its action is to make the next element of the `ResultSet` object the current one—that is, the one that can be accessed through one of the access methods provided by `ResultSet`. Initially, there is no current element of a `ResultSet` object. Therefore, `next` must be called to make the first element current. The elements of a `ResultSet` object are typically accessed in a loop such as the following:

```
while(result.next()) {
     access and process the current element
}
```

Here, `result` is the object returned by `executeQuery`.

The actual structure of a `ResultSet` object is not visible to the application, so it is irrelevant. The information in a `ResultSet` object is extracted through a collection of access methods. Each element of a `ResultSet` object represents the information in a row of the result of the query operation. Field values in the rows can be extracted by the access methods, whose names are in the following general form:

`get`*Type_name*

Here, the *Type_name* part is one of the Java data types, either a primitive type such as `int` or `float` or a class such as `String`.

There are actually two of each of the named access methods: one that takes an `int` parameter, which specifies the column number, starting at `1`; and one that takes a `String` parameter, which specifies the column name. For example, suppose the first row of the `ResultSet` object for the `SELECT` specified previously happened to be as follows:

```
3, "convertible", 13.5, 2001, 1
```

Assuming that the variable `style` is defined to be a `String` object, the value of the `Body_style` column `"convertible"` could be obtained with either of the following two method calls:

```
style = result.getString("Body_style");
style = result.getString(2);
```

The SQL data types do not perfectly match the Java data types. Some of the most commonly used SQL data types and their Java counterparts are shown in Table 13.1.

Table 13.1 Common SQL data types and their Java counterparts

SQL Data Type	Java Data Type
`INTEGER` or `INT`	`int`
`SMALLINT`	`short`
`FLOAT(`*n*`)`	`double`
`REAL`	`float`
`DOUBLE`	`double`
`CHARACTER(`*n*`)` or `CHAR(`*n*`)`	`String`
`VARCHAR(`*n*`)`	`String`
`BOOLEAN`	`boolean`

The `get`*Type_name* methods attempt to convert SQL data types to equivalent Java data types. For example, if `getString` is used to fetch an `INTEGER` value, the number will be converted to a `String` object.

13.7.2 Metadata

If a servlet is being developed that must work with any database—that is, the exact structure of the database is not known—the code must be able to get table names and column names from the database. Also, the types of the data in the result rows must be known. Information that describes the database itself or some part of the database is called *metadata*. There are two kinds of metadata: metadata that describes the database and metadata that describes a `ResultSet` object that is returned by the execution of a query.

A method of the `Connection` object, `getMetaData`, creates an object of `DatabaseMetaData` type, which can be used to get information about a database. For example:

```
DatabaseMetaData dbmd = myCon.getMetaData();
```

To deal with the many different configurations of databases, many different methods are defined in the `DatabaseMetaData` class. Fortunately, most of them are infrequently used, and we can illustrate the use of metadata through

just one that is commonly used, `getTables`. Although `getTables` returns a variety of information, here we are interested only in table names.

The `getTables` method takes four parameters, only the last of which interests us. The last actual parameter to `getTables` specifies an array of `String` objects with just one element, which is set to the value `"TABLE"`. The other three actual parameters can be `null`. The `getTables` method returns a `ResultSet` object that has information about the tables of the database, the third row of which has the table names. Assuming that the `Connection` object for a database is `myCon`, the code to produce a list of the names of the tables in the database is as follows:

```
String tbl[] = {"TABLE"};
DatabaseMetaData dbmd = myCon.getMetaData();
result = dbmd.getTables(null, null, null, tbl);
System.out.println("The tables in the database are: \n\n");
while (result.next()) {
    System.out.println(result.getString(3));
}
```

Adding this code to a program with access to the `cars` database would produce the following output:

```
The tables in this database are:
CORVETTES
CORVETTES_EQUIPMENT
EQUIPMENT
STATES
```

Fetching metadata about the result of a query on a database is more complicated than getting the table names. The metadata for a query result has a different structure than that for the general database information. For the query result, the metadata is stored in an object of the `ResultSetMetaData` class. An object of this class is returned from the `ResultSet` object when the `getMetaData` method is called, as shown in the following example:

```
ResultSetMetaData resultMd = result.getMetaData();
```

Using the `resultMd` object, the number of columns and their names, types, and sizes can be determined through the methods of `ResultSetMetaData`. The number of columns is returned by `getColumnCount`. The name of the *i*th column is returned by `getColumnLabel(i)`.

Using these objects and methods, the following code creates a display of the column names of the result produced by a query.

```
// Create an object for the metadata
ResultSetMetaData resultMd = result.getMetaData();

// Loop to fetch and display the column names
for (int i = 1; i <= resultMd.getColumnCount(); i++) {
```

```
        String columnName = resultMd.getColumnLabel(i);
        System.out.print(columnName + "\t");
    }
    System.out.println("\n");
```

The display produced by this code is as follows:

```
Vette_id     Body_style      Miles   Year    State
```

The problem of not knowing the types of the data in the result rows has a simple solution: The data can be converted to strings with `getString`, a method of the result object. This is illustrated in Section 13.7.3.

13.7.3 An Example

As an example, an HTML document that collects a database query in a text box will be used, similar to `carsdata.html`, which is used in Section 13.6.4 as the user interface to the PHP/MySQL example. The document calls a servlet to perform the query. The servlet uses its `init` method to establish the database connection and create the `Statement` object for the query method, `executeQuery`. These operations could be specified in the `doPost` method, but that would require reconnection to the database with every query. In the `init` method, they only happen once.

The `doPost` method performs the query operations and builds the return document of the results of the query. The query results are placed in an HTML table so that the output has a presentable appearance.

```
// JDBCServlet.java
//  This servlet receives an SQL query from its HTML document,
//   connects to the cars database, performs the query on the
//   database, and returns an HTML table of the results of the
//   query

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import java.util.*;
import java.sql.*;

public class JDBCServlet extends HttpServlet {
  private Connection myCon;
  private Statement myStmt;

// The init method - instantiate the db driver, connect to the
//  db, and create a statement for an SQL command
  public void init() {
```

```
// Instantiate the driver for MySQL
    try {
      Class.forName("org.gjt.mm.mysql.Driver").newInstance();
    }
    catch (Exception e) {
      e.printStackTrace();
    }

// Create the connection to the cars db
    try {
      myCon = DriverManager.getConnection (
              "jdbc:mysql://localhost/cars?user=root");
    }
    catch (SQLException e) {
      e.printStackTrace();
    }

// Create the statement for SQL queries
    try {
      myStmt = myCon.createStatement();
    }
    catch (Exception e) {
      e.printStackTrace();
    }
  }  //** end of the init method

// The doPost method - get the query, perform it, and produce
//  an HTML table of the results
  public void doPost(HttpServletRequest request,
                     HttpServletResponse response)
                throws ServletException, IOException {
    ResultSet result;
    String query, colName, dat;
    int numCols, index;
    ResultSetMetaData resultMd;

// Get the SQL request command
    query = request.getParameter("Query");

// Set the MIME type and get a writer
    response.setContentType("text/html");
    PrintWriter out = response.getWriter();

// Create the initial html and display the request
    out.println("<html>");
```

```
    out.println("<head><title>JDBCServlet</title></head>");
    out.println("<body>");
    out.print("<p><b>The query is: </b>" + query + "</p>");

// Perform the query
    try {
      result = myStmt.executeQuery(query);

// Get the result's metadata and the number of result rows
      resultMd = result.getMetaData();
      numCols = resultMd.getColumnCount();

// Produce the table header and caption
      out.println("<table border>");
      out.println("<caption> <b> Query Results </b> </caption>");
      out.println("<tr>");

// Loop to produce the column headings
      for (index = 1; index <= numCols; index++) {
        colName = resultMd.getColumnLabel(index);
        out.print("<th>" + colName + "</th>");
      }
      out.println("</tr>");

// Loop to produce the rows of the result
      while (result.next()) {
        out.println("<tr>");

// Loop to produce the data of a row of the result
        for (index = 1; index <= numCols; index++) {
          dat = result.getString(index);
          out.println("<td>" + dat + "</td>");
        }  //** end of for (index = 0; ...
        out.println("</tr>");
      }  //** end of while (result.next()) ...
    out.println("</table>");
    }  //** end of try

    catch (Exception e) {
      e.printStackTrace();
    }  //** end of catch
    out.println("</body></html>");
  }  //** end of doPost method
}  //** end of class JDBCServlet
```

Figure 13.10 shows a browser display of the output of `JDBCServlet` on a given query.

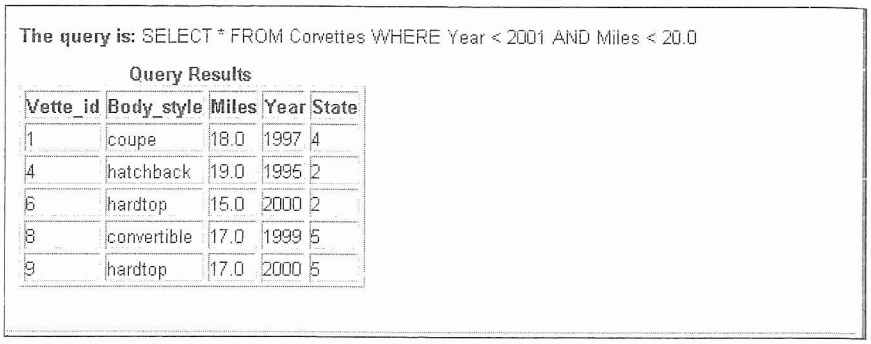


Figure 13.10 Display of the results of `JDBCServlet`

Summary

A relational database consists of a collection of related tables of data. Most tables include a column of primary keys, which uniquely identify the rows. A cross-reference table contains no data; instead, it contains the primary keys of two data tables, providing a many-to-many relationship between the data in the two tables.

SQL is a standard language for specifying accesses and modifications to relational databases. All commonly used relational database systems support SQL. The most frequently used SQL commands are `CREATE`, `SELECT`, `INSERT`, `UPDATE`, and `DELETE`.

The `CREATE` command specifies a table name and a list of column names and their associated constraints. The `SELECT` command specifies one or more columns of one or more tables, along with a Boolean expression that provides a constraint on the data in the specified columns. `SELECT` is a complex and powerful tool. The `INSERT` command specifies a table name, a list of column names, and a list of values that correspond to the column names. The `UPDATE` command specifies a table name and a list of column name/value pairs, along with a specific primary key value. The `DELETE` command specifies a table name and the primary key of a specific column.

A join operation, which can be specified by a `SELECT` command, creates a new table by joining part of the data of one table with part of the data of another table. The objective of a join is to make information available to the user that is not stored in a single table.

A two-tier client/server architecture is common. In it, a client machine communicates directly with a server machine. The Web is an example of a two-tier client/server configuration. A third tier is used in a client/server architec-

ture when it is better for one or both of the client or the server to communicate only indirectly with the other.

One approach to building database applications is to extend a general-purpose programming language so that it can specify SQL commands and interact with a database through them. The disadvantage of this is that such applications are not likely to be portable among the databases of different vendors. Microsoft's Access system provides a way to access the databases of most common vendors through an interface called ODBC. Because ODBC has been implemented by most vendors for their databases, this approach provides a way to develop portable applications.

MySQL is a relational database server that implements SQL. There are drivers for MySQL for most common database APIs, including Perl DBI, PHP, and JDBC.

Perl's DBI module provides an object through which Perl programs can connect to databases and use SQL to perform operations. SQL commands are often precompiled with the `prepare` method. Such commands are executed with the `execute` method. Column names for query results are obtained through the `NAME` attribute of the `DBI` object.

PHP has implemented APIs for most common database systems. The MySQL API for PHP includes functions for connecting to a database (`mysql_connect`), executing SQL commands (`mysql_query`), and retrieving rows from query results (for example, `mysql_fetch_array`). Getting the column names for query results is a bit confusing but not difficult.

The goal of JDBC is related to that of ODBC, except that it is part of one general-purpose programming language, Java. There are drivers for JDBC for all common database systems. A servlet must create a connection to a database for which there is a JDBC driver available. Then it creates a `Statement` object into which can be stored an SQL command as a string. The command can be executed by passing it as a parameter to a method through the `Statement` object. The return value from the execution of a `SELECT` command is an object of `ResultSet` type, which stores the rows that were extracted from the database. Actual data values are obtained from the returned object through a collection of methods called through the object.

Metadata is data about the database rather than data stored in the database. It is common to need information about the result object returned from the execution of a `SELECT` command. This information is obtained by a method called through the result object. Specific information is obtained by methods called through the metadata object.

Review Questions

13.1 What is the purpose of the primary keys of a table in a relational database?

13.2 What is the purpose of a cross-reference table?

13.3 How are string literals delimited in SQL?

13.4 What does the `NOT NULL` constraint specify in a column of a `CREATE TABLE` SQL command?

13.5 What does an asterisk specify when it appears as the value of a `SELECT` clause?

13.6 What is specified by the `WHERE` clause of a `SELECT` command?

13.7 How are the column names associated with the values in an `INSERT` command?

13.8 What is the purpose of an `UPDATE` command?

13.9 What exactly is a table join, and how is one specified in SQL?

13.10 What is the purpose of a third tier in a client/server configuration for Web access to a database?

13.11 Why are two-tier client/server configurations sometimes inadequate?

13.12 Explain how SQL database access can be provided by extending a programming language.

13.13 What is the disadvantage of embedding SQL in a programming language?

13.14 What is ODBC, and why is it useful?

13.15 What does the Perl DBI module provide?

13.16 What is the relationship between ODBC and JDBC?

13.17 What is MySQL?

13.18 What does the MySQL constraint `auto_increment` do?

13.19 What is the form of the first parameter to the Perl DBI method `connect`?

13.20 What does the Perl DBI `prepare` method do?

13.21 What is a Perl DBI statement handle?

13.22 What is the problem with quotes in an SQL command obtained from a form element in an HTML document?

13.23 What is the purpose of the PHP `mysql_select_db` function?

13.24 How can a PHP program determine the number of rows in a query result?

13.25 What does the PHP function `mysql_fetch_array` do?

13.26 Explain the exact form of the value returned by `mysql_fetch_array`.

13.27 Explain the two ways of using JDBC.

13.28 What advantage does a third-tier computer provide when using JDBC?

13.29 What method of what class is used to connect to a database when using JDBC?

13.30 Explain the two ways to register a JDBC driver.

13.31 What purpose does a `Statement` object serve when using SQL through JDBC?

13.32 What method of what class is used to execute an SQL action command?

13.33 What method of what class is used to execute a `SELECT` command?

13.34 What class of object is returned from the `executeQuery` method?

13.35 How can a program iterate through the object returned by `executeQuery`?

13.36 What is the form of the methods used to extract values from the object returned by `executeQuery`?

13.37 What is metadata?

13.38 How is the collection of metadata extracted from a database?

13.39 What are the two ways column labels can be obtained from an object of metadata?

Exercises

13.1 Use MySQL to create a database of information about used trucks for sale, similar to the `cars` database used in this chapter. Make up equipment that characterizes trucks. Get the raw data from the ad section of your local newspaper. Instead of the states in the `cars` database, divide your town into four sections and use them.

13.2 Modify and test the program `access_cars.pl` to handle `UPDATE` and `INSERT` SQL commands.

13.3 Modify and test the program `access_cars.php` to handle `UPDATE` and `INSERT` SQL commands.

13.4 Modify and test the program `JDBCServlet.java` to handle `UPDATE` and `INSERT` SQL commands.

CHAPTER

14

Introduction to Ruby

14.1 Origins and Uses of Ruby
14.2 Scalar, Types and Their Operations
14.3 Simple Input and Output
14.4 Control Statements
14.5 Fundamentals of Arrays
14.6 Hashes
14.7 Methods
14.8 Classes
14.9 Code Blocks and Iterators
14.10 Pattern Matching
Summary • *Review Questions* • *Exercises*

Our primary interest in Ruby in this book is its use with the Web software development framework, Rails. However, Ruby is an interesting and useful language outside its use in Rails. This chapter takes you on a quick tour of Ruby, introducing many of the important concepts and constructs but leaving out many of the details of the language. In spite of its brevity, however, if you are an experienced programmer, particularly one well-versed in object-oriented programming, you can learn to write useful Ruby programs by studying this chapter. However, be warned that Ruby differs fundamentally from conventional languages, such as C++, Java, and C#, both in its syntax and because it is an interpreted scripting language. If you need more details than can be found in this chapter, there are several books dedicated solely to Ruby, as well as a Web

site that includes a wide variety of information about the language (`www.ruby-lang.org`).

This chapter begins with a description of Ruby's scalar data types and their use in expressions and assignment statements. Next, it covers control expressions and the collection of control constructs available in Ruby. It then discusses Ruby's two built-in data structures, arrays and hashes. This is followed by a description of methods and how they are defined and called. Next, some of the details of classes are introduced. Lastly, code blocks, iterators, and pattern matching are described. Although we attempt to introduce a significant subset of Ruby in a single chapter, do not be misled into thinking that this is a small or simple language—it is neither.

14.1 Origins and Uses of Ruby

As stated in Chapter 1, "Fundamentals," Ruby was designed in Japan by Yukihiro Matsumoto (a.k.a. Matz) and was released in 1996. It started as a replacement for the languages Matz found inadequate for his purposes, Perl and Python. Use of Ruby in Japan grew rapidly and spread to the rest of the world a few years later. The quick growth of the use of Rails, the Web application development framework that is both written in Ruby and uses Ruby, has accelerated the expansion of use of the language. Rails is probably the most common use of Ruby.

Learning Ruby is made easier by its implementation method, pure interpretation. Rather than needing to learn about and write a layer of boilerplate code around some simple logic, in Ruby one can write just that simple logic and request its interpretation. For example, consider the difference between a "Hello World" program in a language like C++ or Java with the Ruby "Hello World" program:

```
puts "Hello World"
```

From Perl, Ruby gets regular expressions and implicit variables. From JavaScript it gets objects that can change during execution. However, Ruby has many more differences with those languages than it has similarities. For example, as in pure object-oriented languages, every data value in Ruby is an object, whether it is a simple integer literal or a complete file system.

Ruby is available for every common computing platform. Furthermore, as is the case for Perl and PHP, the Ruby implementation is free.

14.2 Scalar Types and Their Operations

Ruby has three categories of data types—scalars, arrays, and hashes. This section discusses the important characteristics of the most commonly used types, namely, scalars. There are two categories of scalar types, numerics and character strings.

As stated earlier, everything in Ruby is an object—numeric literals, arrays, and even classes. Although this design is much more elegant than the mixed-type design of Java and C++, it takes a bit of getting used to.

14.2.1 Numeric and String Literals

All numeric data types in Ruby are descendants of the `Numeric` class. The immediate child classes of `Numeric` are `Float` and `Integer`. The `Integer` class has two child classes, `Fixnum` and `Bignum`.

An integer literal that fits into the range of a machine word, which is often 32 bits, is a `Fixnum` object. An integer literal that is outside the `Fixnum` range is a `Bignum` object. Though it is odd among programming languages, there is no length limitation (other than your computer's memory size) on integer literals. If a `Fixnum` integer grows beyond size limitation of `Fixnum` objects, it is coerced to a `Bignum` object. Likewise, if an operation on a Bignum object results in a value that fits in a `Fixnum` object, it is coerced to a `Fixnum` type.

Underscore characters can appear embedded in integer literals. Ruby ignores such underscores. This allows large numbers to be slightly more readable. For example, instead of `124761325`, `124_761_325` can be used.

A numeric literal that has either an embedded decimal point or a following exponent is a `Float` object, which is stored as the underlying machine's double-precision floating-point type. The decimal point must be embedded; that is, it must be both preceded and followed by at least one digit. So, `.435` is not a legal literal in Ruby.

All string literals are `String` objects, which are sequences of bytes that represent characters. Ruby's string literals are related to those of Perl in that there are two categories, single quoted and double quoted. Single-quoted string literals cannot include characters specified with escape sequences, such as newline characters specified with `\n`. If an actual single-quote character is needed in a string literal that is delimited by single quotes, the embedded single quote is preceded by a backslash, as in the following example:

```
'I\'ll meet you at O\'Malleys'
```

If an escape sequence is embedded in a single-quoted string literal, each character in the sequence is taken literally as itself. For example, the sequence `\n` in the following string literal will be treated as two characters, a backslash and an `n`:

```
'Some apples are red, \n some are green'
```

If a string literal with the same characteristics as single-quoted strings is needed but you want to use a different delimiter, precede the delimiter with `q`, as in the following example:

```
q$Don't you think she's pretty?$
```

If the new delimiter is a parenthesis, a brace, a bracket, or a pointed bracket, the left element of the pair must be used on the left, and the right element must be used on the right. For example:

```
q<Don't you think she's pretty?>
```

Double-quoted string literals differ from single-quoted string literals in two ways: First, they can include special characters specified with escape sequences; second, the values of variable names can be interpolated into the string, which means that their values are substituted for their names. We discuss the first of these differences here; the other will be discussed in Section 14.2.2.

In many situations, special characters that are specified with escape sequences must be included in string literals. For example, if the words on a line must be spaced by tabs, a double-quoted literal with embedded escape sequences for the tab character can be used as in the following:

```
"Runs \t Hits \t Errors"
```

A double quote can be embedded in a double-quoted string literal by preceding it with a backslash.

A different delimiter can be specified for string literals with the characteristics of double-quoted strings by preceding the new delimiter with `Q` as follows:

```
Q@"Why not learn Ruby?", he asked.@
```

The null string (one with no characters) can be denoted with either `''` or `""`.

14.2.2 Variables and Assignment Statements

Naming conventions in Ruby help identify different categories of variables. For now, we will deal with only local variables. We will explain other naming conventions when they are needed.

The form of an identifier is a lowercase letter or an underscore, followed by any number of uppercase or lowercase letters, digits, or underscores. The letters in a variable name are case sensitive, meaning that `fRIZZY`, `frizzy`, `frIzZy`, and `friZZy` are all distinct names. However, by convention, programmer-defined variable names do not include uppercase letters.

As mentioned earlier, double-quoted string literals can include the values of variables. In fact, the results of executing any Ruby code can be included. This is specified by placing the code in braces and preceding the left brace with a pound sign (#). For example:

```
"Tuesday's high temperature was #{tue_high}"
```

If the value of `tue_high` is `83`, this string has the following value:

```
"Tuesday's high temperature was 83"
```

Consider the following literal:

```
"The cost of our apple order is $#{price * quantity}"
```

If the value of price is `1.65` and quantity is `6`, the value of this string is

```
"The cost of our apple order is $9.90"
```

Because Ruby is a pure object-oriented programming language, its variables are all references to objects. This is in contrast to more conventional languages, such as C++ and Java, which have two categories of variables, those for primitives and those that reference objects. In Ruby, every data value is an object, so it needs only references. Because references in Ruby are typeless, there is no point in declaring them. In fact, there is no way in Ruby to declare a variable.

A scalar variable that has not been assigned a value by the program has the value `nil`.

Ruby has constants, which are distinguished from variables by their names, which always begin with uppercase letters. A constant is created when it is assigned a value, which can be any constant expression. In Ruby, a constant can be assigned a new value, although it causes a warning message to the user.

Ruby includes some predefined, or *implicit*, variables. The names of implicit scalar variables begin with dollar signs. The rest of the name of an implicit variable is often just one more special character, such as an underscore (`_`), a circumflex (`^`), or a backslash (\). This chapter and the next include some uses of these implicit variables.

The assignment statements of Ruby are exactly like those of the programming languages derived from C. The only thing to remember is that the variables of Ruby are all typeless references. All that is ever assigned in an assignment statement is the address of an object.

14.2.3 Numeric Operators

Most of Ruby's numeric operators are similar to those in other common programming languages, so they should be familiar to most readers. These are the binary operators `+` for addition, `-` for subtraction, `*` for multiplication, `/` for division, `**` for exponentiation, and `%` for modulus. The modulus operator is defined as follows: `x % y` produces the remainder of the value of `x` after division by `y`. If an integer is divided by an integer, integer division is done. Therefore, `3 / 2` produces `1`.

The precedence rules of a language specify which operator is evaluated first when two operators that have different levels of precedence appear in an expression, separated only by an operand. The associativity rules of a language specify which operator is evaluated first when two operators with the same precedence level appear in an expression, separated only by an operand. The precedence and associativity of the numeric operators are given in Table 14.1.

Note that Ruby does not include the increment (`++`) and decrement (`--`) operators found in all of the C-based languages.

Ruby includes the `Math` module, which has methods for basic trigonometric and transcendental functions. Among these are `cos` (cosine), `sin` (sine), `log` (logarithm), `sqrt` (square root), and `tan` (tangent). The methods of the `Math`

Table 14.1 Precedence and associativity of the numeric operators

Operator	Associativity
**	Right
+, –	Right
*, /, %	Left
binary +, –	Left

The operators listed first have the highest precedence.

module are referenced by prefixing their names with `Math.`, as in `Math.sin(x)`. All of these take any numeric type as a parameter and return a `Float` value.

Included with the Ruby implementation is an interactive interpreter, which is very useful to the student of Ruby. It allows one to type any Ruby expression and get an immediate response from the interpreter. The interactive interpreter's name is *Interactive Ruby*, whose acronym, irb, is the name of the program that supports it. `irb` is entered by simply typing `irb` at the command prompt in the directory that contains the Ruby interpreter. For example, if the command prompt is a percent sign (`%`), one can type

```
% irb
```

`irb` will respond with its own prompt, which is

```
irb(main):001:0>
```

At this prompt, any Ruby expression or statement can be typed. `irb` interprets the expression or statement and returns the value after an implication symbol (=>). For example:

```
irb(main):001:0> 17 * 3
=> 51
irb(main):002:0>
```

The lengthy default prompt can be easily changed. We prefer the simple ">>" prompt. The default prompt can be changed to this with the following command:

```
irb(main):002:0> conf.prompt_i = ">>"
```

From here on, we will use this simple prompt.

14.2.4 String Methods

The Ruby `String` class has more than 75 methods, a few of which are described in this section. Many of these methods can be used as if they were

operators. In fact, we sometimes call them operators, even though underneath they are all methods.

The `String` method for catenation is specified by plus (+), which can be used as a binary operator. This method creates a new string from its operands. For example:

```
>> "Happy" + " " + "Holidays!"
=> "Happy Holidays!"
```

To append a string to the right end of another string, which of course only makes sense if the left operand is a variable, use the << method. Like +, the << method can be used as a binary operator. For example:

```
>> mystr = "G'day "
=> "G'day "
>> mystr << "mate"
=> "G'day mate"
```

The first assignment above creates the specified string literal and sets the variable `mystr` to reference that memory location. If `mystr` is assigned to another variable, that variable will reference the same memory location as `mystr`. For example:

```
>> mystr = "Wow!"
=> "Wow!"
>> yourstr = mystr
=> "Wow!"
>> yourstr
=> "Wow!"
```

Now, both `mystr` and `yourstr` reference the same memory location, the place that has the string `"Wow!"`. If a different string literal is assigned to `mystr`, Ruby will build a memory location with the value of the new string literal and `mystr` will reference that location. But `yourstr` will still reference the location with `"Wow!"`. For example:

```
>> mystr = "Wow!"
=> "Wow!"
>> yourstr = mystr
=> "Wow!"
>> mystr = "What?"
=> "What?"
>> yourstr
=> "Wow!"
```

If you want to change the value of the location that `mystr` references, but let `mystr` reference the same memory location, the `replace` method is used. For example:

```
>> mystr = "Wow!"
=> "Wow!"
>> yourstr = mystr
=> "Wow!"
>> mystr.replace("Golly!")
=> "Golly!"
>> mystr
=> "Golly!"
>> yourstr
=> "Golly!"
```

Now, `mystr` and `yourstr` still reference the same memory location.

The append operation can also be done with the += assignment operator. So, instead of `mystr << "mate"`, `mystr += "mate"` could be used.

In the following paragraphs, other string functions will be introduced that also change a string value but leave the affected variable referencing the same memory location.

The other most commonly used methods are similar to those of other programming languages. Among these are the following, all of which create new strings:

Method	Action
`capitalize`	Convert the first letter to uppercase and the rest of the letters to lowercase
`chop`	Removes the last character
`chomp`	Removes a newline from the right end, if there is one
`upcase`	Converts all of the lowercase letters in the object to uppercase
`downcase`	Converts all of the uppercase letters in the object to lowercase
`strip`	Removes the spaces on both ends
`lstrip`	Removes the spaces on the left end
`rstrip`	Removes the spaces on the right end
`reverse`	Reverses the characters of the string
`swapcase`	Convert all uppercase letters to lowercase and all lowercase letters to uppercase

As stated previously, all of these produce new strings, rather than modify the given string in place. However, all of these methods have versions that do modify their objects in place. These are called *bang* or *mutator* methods, which are specified by following their names with an exclamation point (`!`). To illustrate the difference between a string method and its bang counterpart, consider the following interactions:

```
>> str = "Frank"
=> "Frank"
>> str.upcase
=> "FRANK"
```

```
>> str
=> "Frank"
>> str.upcase!
=> "FRANK"
>> str
=> "FRANK"
```

Note that after using `upcase`, the value of `str` is unchanged (it is still `"Frank"`), but after using `upcase!`, it is changed (it is `"FRANK"`).

Ruby strings can be indexed, somewhat as if they were arrays. As one would expect, the indices begin at zero. The brackets of this method specify a getter method. The catch with this getter method is that it returns the ASCII code (as a `Fixnum` object), rather than the character. To get the character, the `chr` method must be called. Consider the following example:

```
>> str = "Shelley"
=> "Shelley"
>> str[3]
=> 108
>> str[3].chr
=> "l"
```

If a negative subscript is used as an index, the position is counted from the right.

A multicharacter substring of a string can be accessed by including two numbers in the brackets, in which case the first is the position of the first character of the substring and the second is the number of characters in the substring. Unlike the single-character reference, however, in this case the value is a string, not a number. For example:

```
>> str = "Shelley"
=> "Shelley"
>> str[2,4]
=> "elle"
```

The substring getter can be used on individual characters to get one character without calling the `chr` method.

Specific characters of a string can be set with the set method, `[]=`. For example:

```
>> str = "Donald"
=> "Donald"
>> str[3,3] = "nie"
=> "nie"
>> str
=> "Donnie"
```

The usual way to compare strings for equality is to use the == method as an operator. For example:

```
>> "snowstorm" == "snowstorm"
=> true
>> "snowie" == "snowy"
=> false
```

A different sense of equality is tested with the `equal?` method, which determines whether its parameter is the same object as the one to which it is sent. For example:

```
>> "snowstorm".equal?("snowstorm")
=> false
```

This produces `false` because, although the contents of the two string literals are the same, they are different objects.

Yet another sense of equality is tested with the `eql?` method. It returns `true` if its receiver object and its parameter have the same types and the same values. For example:

```
>> 7 == 7.0
=> true
>> 7.eql?(7.0)
=> false
```

To facilitate ordering, Ruby includes the "spaceship" operator, `<=>`. It returns `-1` if the second operand is greater than the first, `0` if they are equal, and `1` if the first operand is greater than the second. Greater in this case means it belongs later in alphabetical order. For example:

```
>> "apple" <=> "prune"
=> -1
>> "grape" <=> "grape"
=> 0
>> "grape" <=> "apple"
=> 1
```

The repetition operator is specified with an asterisk (`*`). It takes a string as its left operand and an expression that evaluates to a number as its right operand. The left operand is replicated the number of times equal to the value of the right operand. For example:

```
>> "More! " * 3
=> "More! More! More! "
```

14.3 Simple Input and Output

Among the most fundamental constructs in most programming languages are the statements or functions that provide keyboard input and screen output. The next sections introduce these as they appear in Ruby.

14.3.1 Screen Output

Output is directed to the screen with the `puts` method (or operator). We prefer to treat it as an operator. The operand for `puts` is a string literal. A newline character is implicitly appended to the string operand. If the value of a variable is to be part of a line of output, the `#{...}` notation can be used to insert it in a double-quoted string literal, as in the following:

```
>> name = "Fudgy"
=> "Fudgy"
>> puts "My name is #{name}"
My name is Fudgy
=> nil
```

Notice that the value returned by `puts` is `nil`, and that the value is returned after the string has been displayed.

The `print` method is used if you do not want the implied newline that `puts` puts on the end of your literal string.

14.3.2 Keyboard Input

Because we will use Ruby primarily for Rails, keyboard input is rarely needed. However, keyboard input is certainly useful for other applications, so we briefly introduce it here.

The `gets` method gets a line of input from the keyboard. The retrieved line includes the newline character. If the newline is not needed, it can be discarded with `chomp`. For example:

```
>> name = gets
apples
=> "apples\n"
>> name = name.chomp
=> "apples"
```

This could be shortened by applying `chomp` directly to the value returned by `gets`, as in the following:

```
>> name = gets.chomp
apples
=> "apples"
```

If a number is to be input from the keyboard, the string from `gets` must be converted to an integer with the `to_i` method, as in the following:

```
>> age = gets.to_i
27
=> 27
```

If the number is a floating-point value, the conversion method is `to_f`, as in the following:

```
>> age = gets.to_f
27.5
=> 27.5
```

In this same vein, we must mention that there is a similar method, `to_s`, to which every object responds. It converts the value of the object to which it is sent to a string. However, because `puts` implicitly converts its operand to a string, `to_s` is not often explicitly called.

The following listing is of a trivial program created with a text editor and stored in a file:

```
# quadeval.rb – A simple Ruby program
#  Input: Four numbers, representing the values of
#         a, b, c, and x
# Output: The value of the expression
#          a*x**2 + b*x + c
# Get input
puts "Please input the value of a "
a = gets.to_i
puts "Please input the value of b "
b = gets.to_i
puts "Please input the value of c "
c = gets.to_i
puts "Please input the value of x "
x = gets/to_i
# Compute and display the result
result = a * x ** 2 + b * x + c
puts "The value of the expression is: #{result}"
```

A program stored in a file can be run by the command

```
>ruby filename
```

So, our example can be run (interpreted) with

```
>ruby quadeval.rb
```

If you want a compilation without the interpretation, just to check the syntactic correctness of your program, include the `-c` flag after the `ruby` command. It is also a good idea to include the `-w` flag, which causes `ruby` to produce warning messages for a variety of suspicious things it may find in your program. For example, to check the syntax of our example program, the following could be used:

```
>ruby -cw quadeval.rb
```

If the program is found to be okay, the response to this command is

```
Syntax OK
```

14.4 Control Statements

Ruby has a complete collection of statements for controlling the execution flow through its programs. This section introduces the control expressions and control statements of Ruby.

14.4.1 Control Expressions

The expressions upon which statement control flow is based are Boolean expressions. They can be either of the constants `true` or `false`, variables, relational expressions, or compound expressions. A control expression that is a simple variable is true if its value is anything except `nil` (it references some object). If its value is `nil`, it is false.

A relational expression has two operands and a relational operator. Relational operators can have any scalar-valued expression for their operands. The relational operators are shown in Table 14.2.

Table 14.2 Relational operators

Operator	Operation
==	Is equal to
!=	Is not equal to
<	Is less than
>	Is greater than
<=	Is less than or equal to
>=	Is greater than or equal to
<=>	Compare, returning `-1`, `0`, or `+1`
`eql?`	True if the receiver object and the parameter both have the same type and equal values
`equal?`	True if the receiver object and the parameter have the same object ID

Recall that the <=> operator is often used for comparing strings. Also, `equal?` is used to determine whether two variables are aliases, that is, they reference the same object.

Ruby has two sets of operators for the AND, OR, and NOT Boolean operations. The two sets have the same semantics but different precedence levels.

The operators with the higher precedence are `&&` (AND), `||` (OR), and `!` (NOT). Those with the lower precedence are `and`, `or`, and `not`. The precedence of these latter operators is lower than any other operators in Ruby, so no matter what operators appear in their operands, these operators will be evaluated last.

All of the relational operators are methods, but all except `eql?` and `equal?` can be used as operators.

The precedence and associativity of all operators discussed so far in this chapter are shown in Table 14.3.

Table 14.3 Operator precedence and associativity

Operator	Associativity
`**`	Right
`!`, unary + and -	Right
`*`, `/`, `%`	Left
`+`, `-`	Left
`&`	Left
`+`, `-`	Left
`>`, `<`, `>=`, `<=`	Nonassociative
`==`, `!=`, `<=>`	Nonassociative
`&&`	Left
`||`	Left
`=`, `+=`, `-=`, `*=`, `**=`, `/=`, `%=`, `&=`, `&&=`, `||=`	Right
`not`	Right
`or`, `and`	Left

Highest-precedence operators are listed first.
The method names for unary minus and plus are `-@` and `+@`, respectively.

Because assignment statements have values (the value of an assignment is the value assigned to the left-side variable), they can be used as control expressions. One common use of this is for an assignment statement that reads a line of input. The `gets` method returns `nil` when it gets the end-of-file (EOF) character, so this can be conveniently used to terminate loops. For example:

```
while (next = gets) { … }
```

The keyboard EOF character is Ctrl+D for UNIX, Ctrl+Z for Windows, and ⌘+. for Macintosh systems.

14.4.2 Selection and Loop Statements

Control statements require some syntactic container for sequences of statements whose execution they are meant to control. The Ruby form of such containers is to use a simple sequence of statements terminated with `else` (if the sequence is a then clause) or `end` (if the sequence is either an else clause or it is a then clause and there is no else clause). A *control construct* is a control statement and the segment of code whose execution it controls.

Ruby's `if` statement is similar to that of other languages. One syntactic difference is that there are no parentheses around the control expression, as is the case with most of the languages based directly or even loosely on C. For example, consider the following example:

```
if a > 10
  b = a * 2
end
```

An `if` construct can include `elsif` (note that it is *not* spelled "elseif") clauses, which provide a way of having a more readable sequence of nested `if` constructs. For example:

```
if snowrate < 1
  puts "Light snow"
elsif snowrate < 2
  puts "Moderate snow"
else
  puts "Heavy snow"
end
```

Ruby has an `unless` statement, which is the same as its `if` statement except that the inverse of the value of the control expression is used. This is convenient if you want a selection construct with an else clause but no then clause. The following construct illustrates an `unless` statement:

```
unless sum > 1000
    puts "We are not finished yet!"
end
```

Ruby includes two kinds of multiple selection constructs, both named case. One Ruby case construct, which is similar to a switch, has the following form:

```
case expression
when value then
 - statement sequence
when value then
 - statement sequence
[else
 - statement sequence ]
end
```

The value of the case expression is compared with the when clause values, one at a time from top to bottom until a match is found, at which time the following statement sequence is interpreted. The comparison is done using the === relational operator, which is defined for all built-in classes. If the when value is a range, such as `(1..100)`, === is defined as an inclusive test, yielding true if the value of the case expression is in the given range. If the when value is a class name, === is defined to yield true if the case value is an object of the case expression class or one of its superclasses. If the when value is a regular expression, === is defined to be a simple pattern match. Note that the === operator is only used for the comparisons in case constructs.

Consider the following example:

```
case in_val
when -1 then
  neg_count += 1
when 0 then
  zero_count += 1
when 1 then
  pos_count += 1
else
  puts "Error - in_val is out of range"
end
```

Note that no break statements are needed at the ends of the selectable statement sequences in this construct.

The second form of case constructs uses a Boolean expression to choose a value to be produced by the construct. The general form of this case is as follows:

```
case
when Boolean expression then expression
when Boolean expression then expression
...
when Boolean expression then expression
else expression
end
```

The semantics of this construct is straightforward. The Boolean expressions are evaluated, one at a time until one evaluates to `true`. The value of the whole construct is the value of the expression that corresponds to the true Boolean expression. If none of the Boolean expressions is true, the else expression is evaluated and its value is the value of the construct. For example, consider the following assignment statement:[1]

```
leap = case
       when year % 400 == 0 then true
```

1. This example is from Thomas et al. (2005).

```
when year % 100 == 0 then false
else year % 4 == 0
end
```

This case expression evaluates to true if `year` is a leap year.

The Ruby `while` and `for` statements are similar to those of C and its descendants. The bodies of both are sequences of statements that end with `end`. The general form of the `while` statement is as follows:

`while` *control expression*
    loop body statement(s)
`end`

The `until` statement is similar to the `while` statement except that the inverse of the value of the control expression is used.

For those situations where a loop is needed in which the conditional termination is at some position in the loop other than the top, Ruby has an infinite loop construct and loop exit statements. The body of the infinite loop construct is called a *code block*. Code blocks can appear in two forms, one where the delimiters are braces and one where the delimiters are the reserved words `begin` and `end`. The convention in Ruby is to use the braces when the code block has only one statement and to use `begin/end` when there is more than one statement in the code block.The structure of the unconditional loop is as follows:

`loop`
  code block

There are two ways to control an infinite loop, the `break` and `next` statements. These statements can be made conditional by putting them in the then clause of an `if` construct. The `break` statement causes control to go to the first statement following the code block. The `next` statement causes control to go to the first statement in the code block. For example, consider the following two infinite loop constructs:

```
sum = 0
loop
  begin
  dat = gets.to_i
  if dat < 0 break
  sum += dat
  end
```

```
sum = 0
loop
  begin
  dat = gets.to_i
  if dat < 0 next
  sum += dat
  end
```

In the first construct above, the loop is terminated when a negative value is input. In the second, negative values are not added to `sum`, but the loop continues.

Ruby does not have a general `for` statement, which is ubiquitous among languages with C in their ancestry. However, Ruby includes convenient ways to construct the counting loops implemented with `for` statements in other common languages. These are built with iterator methods, which we postpone discussing until methods and arrays have been introduced. Also, there is a `for` construct in Ruby, but it is used for iterating through arrays and hashes (associative arrays), so once again we postpone the discussion until after arrays and hashes have been introduced (see Section 14.9).

14.5 Fundamentals of Arrays

Ruby includes two structured classes or types, arrays and hashes. Arrays are introduced in this section; hashes are introduced in Section 14.6.

Arrays in Ruby are more flexible than those of most of the other common languages. This is a result of two fundamental differences between Ruby arrays and those of other common languages such as C, C++, and Java. First, the length of a Ruby array is dynamic—it can grow or shrink any time during program execution. Second, a Ruby array can store different types of data. For example, an array may have some numeric elements, some string elements, and even some array elements.

Ruby arrays can be created in two different ways. First, an array can be created by sending the `new` message to the predefined `Array` class, including a parameter for the size of the array. The second way is simply to assign a list literal to a variable, where a list literal is a list of literals delimited by brackets. For example, in the following, the first array is created with `new` and the second is created by assignment:

```
>> list1 = Array.new(5)
=> [nil, nil, nil, nil, nil]
>> list2 = [2, 4, 3.14159, "Fred", [] ]
=> [2, 4, 3.14159, "Fred", []]
```

An array created with the `new` method can also be initialized by including a second parameter, but every element is given the same value (that of the second parameter). For example:

```
>> list1 = Array.new(5, "Ho")
=> ["Ho", "Ho", "Ho", "Ho", "Ho"]
```

Actually, this approach is rarely useful, because not only is each element given the same value, but also each is given the same reference. All of them reference the same object. So, if one is changed, all are changed.

All Ruby array elements use integers as subscripts, and the lower-bound subscript of every array is zero. Array elements are referenced through sub-

scripts delimited by brackets ([ ]), which is actually a getter method that is allowed to be used as a unary operator. Likewise, [ ]= is a setter method. A subscript can be any numeric-valued expression. If an expression with a floating-point value is used as a subscript, the fractional part is truncated. For example:

```
>> list = [2, 4, 6, 8]
=> [2, 4, 6, 8]
>> second = list[1]
=> 4
>> list[3] = 9
=> 9
>> list
=> [2, 4, 6, 9]
>> list[2.999999]
=> 6
```

The length of an array is dynamic; elements can be added or removed from an array using the methods described in Section 14.5.2. The length of an array can be retrieved with the `length` method. For example:

```
>> list.length
=> 4
```

14.5.1 The for-in Statement

The `for-in` statement is used to process the elements of an array. For example, the following code computes the sum of all of the values in `list`:

```
>> sum = 0
=> 0
>> list = [2, 4, 6, 8]
=> [2, 4, 6, 8]
>> for value in list
>>    sum += value
>> end
=> [2, 4, 6, 8]
>> sum
=> 20
```

Notice that the interpreter's response to the `for-in` construct is the values assumed by the scalar variable.

The scalar variable in a `for-in` takes on the values of the `list` array, one at a time. Notice that the scalar *does not* get references to array elements, it gets the values. Consider the following code:

```
>> list = [1, 3, 5, 7]
=> [1, 3, 5, 7]
>> for value in list
>>   value += 2
```

```
>> end
=> [1, 3, 5, 7]
>> list
=> [1, 3, 5, 7]
```

A literal array value can be used in the `for-in` construct, as in the following:

```
>> list = [2, 4, 6]
=> [2, 4, 6]
>> for index in [0, 1, 2]
>> puts "For index = #{index}, the value is #{list[index]}"
>> end
For index = 0, the element is 2
For index = 1, the element is 4
For index = 2, the element is 6
```

14.5.2 Built-In Methods for Arrays and Lists

This section introduces a few of the many built-in methods that are part of Ruby.

Frequently it is necessary to place new elements on one end or the other of an array. Ruby has four methods for this purpose: `unshift` and `shift`, which deal with the left end of arrays; and `pop` and `push`, which deal with the right end of arrays.

The `shift` method removes and returns the first element (lowest subscript) of the array object to which it is sent. For example, the following statement removes the first element of `list` and places it in `first`:

```
>> list = [3, 7, 13, 17]
=> [3, 7, 13, 17]
>> first = list.shift
=> 3
>> list
=> [7, 13, 17]
```

The subscripts of all of the other elements in the array are reduced by 1 as a result of the `shift` operation.

The `pop` method removes and returns the last element from the array object to which it is sent. In this case, there is no change in the subscripts of the array's other elements.

The `unshift` method takes a scalar or an array literal as a parameter. The scalar or array literal is appended to the beginning of the array. This results in an increase in the subscripts of all other array elements. The `push` method also takes a scalar or an array literal. The scalar or array is added to the high end of the array:

```
>> list = [2, 4, 6]
=> [2, 4, 6]
```

```
>> list.push(8, 10)
=> {2, 4, 6, 8, 10]
```

Either `pop` and `unshift` or `push` and `shift` can be used to implement a queue in an array, depending on the direction the queue should grow.

While `push` is a convenient way to add literal elements to an array, if an array is to be catenated to the end of another array, another method, `concat`, is used. For example:

```
>> list1 = [1, 3, 5, 7]
=> [1, 3, 5, 7]
>> list2 = [2, 4, 6, 8]
=> [2, 4, 6, 8]
>> list1.concat(list2)
=> [1, 3, 5, 7, 2, 4, 6, 8]
```

If two arrays need to be catenated together and the result saved as a new array, the plus (+) method can be used as a binary operator, as in the following:

```
>> list1 = [0.1, 2.4, 5.6, 7.9]
=> [0.1, 2.4, 5.6, 7.9]
>> list2 = [3.4, 2.1, 7.5]
=> [3.4, 2.1, 7.5]
>> list3 = list1 + list2
=> [0.1, 2.4, 5.6, 7.9, 3.4, 2.1, 7.5]
```

Note that neither `list1` nor `list2` are affected by the plus method.

The `reverse` method does what its name implies. For example:

```
>> list = [2, 4, 8, 16]
=> [2, 4, 8, 16]
>> list.reverse
=> [16, 8, 4, 2]
>> list
=> [2, 4, 8, 16]
```

Note that `reverse` returns a new array and does not affect the array to which it is sent. The mutator version of `reverse`, `reverse!`, does what `reverse` does, but changes the object to which it is sent. For example:

```
>> list = [2, 4, 8, 16]
=> [2, 4, 8, 16]
>> list.reverse!
=> [16, 8, 4, 2]
>> list
=> [16, 8, 4, 2]
```

The `include?` predicate method searches an array for a specific object. For example:

```
>> list = [2, 4, 8, 16]
```

```
=> [2, 4, 8, 16]
>> list.include?(4)
=> true
>> list.include?(10)
=> false
```

The `sort` method sorts the elements of an array, as long as Ruby knows how to compare those elements. The most commonly sorted elements are either numbers or strings and Ruby knows how to compare numbers with numbers and strings with strings. So, `sort` works well on arrays of elements of either of these two types. For example:

```
>> list = [16, 8, 4, 2]
=> [16, 8, 4, 2]
>> list.sort
=> [2, 4, 8, 16]
>> list2 = ["jo", "fred", "mike", "larry"]
=> ["jo", "fred", "mike", "larry"]
>> list2.sort
=> ["fred", "jo", "larry", "mike"]
```

If the `sort` method is sent to an array that has mixed types, Ruby produces an error message indicating the comparison failed. For example:

```
>> list = [2, "jo", 8, "fred"]
=> [2, "jo", 8, "fred"]
>> list.sort
ArgumentError: comparison of Fixnum with String failed
        from (irb):13:in 'sort'
        from (irb):13
        from :0
```

`sort` returns a new array and does not change the array to which it is sent. The mutator method, `sort!`, sorts the array to which it is sent, in place.

In some situations, arrays represent sets. There are three methods that perform set operations on two arrays. All are used as binary infix operators. They are `&`, for set intersection, `-`, for set difference, and `|`, for set union. Consider the following examples:

```
>> set1 = [2, 4, 6, 8]
=> [2, 4, 6, 8]
>> set2 = [4, 6, 8, 10]
=> [4, 6, 8, 10]
>> set1 & set2
=> [4, 6, 8]
>> set1 - set2
=> [2]
>> set1 | set 2
=> [2, 4, 6, 8, 10]
```

There are a number of other interesting and useful methods that operate on arrays that use blocks. Some of them will be discussed after subprograms and blocks have been introduced.

14.5.3 An Example

The following example illustrates a simple use of an array. A list of names is read from the keyboard. Each name is converted to all uppercase letters and placed in an array. The array is then sorted and displayed.

```
# process_names.rb - A simple Ruby program to
#  illustrate the use of arrays
#  Input: A list of lines of text, where each line
#         is a person's name
# Output: The input names, after all letters are
#         converted to uppercase, in alphabetical order

index = 0
names = Array.new

# Loop to read the names and process them
while(name = gets)

# Convert the name's letters to uppercase and put it
#  in the names array
  names[index] = name.chomp.upcase
  index += 1
end

# Sort the array in place and display it
names.sort!
puts "The sorted array"
for name in names
  puts name
end
```

14.6 Hashes

Associative arrays are arrays in which each data element is paired with a key, which is used to identify the data element. Because hash functions are used to find specific elements in an associative array, associative arrays often are called *hashes*. The two fundamental differences between arrays and hashes are as follows: First, arrays use numeric subscripts to address specific elements, whereas

hashes use string values (the keys) to address elements. Second, the elements in arrays are ordered by subscript, but the elements in hashes are not. In a sense, elements of an array are like those in a list, whereas elements of a hash are like those in a set, where order is irrelevant. The actual arrangement of the elements of a hash in memory is determined by the hash function used to insert and access them.

Like arrays, hashes can be created in two ways, with the new method or by assigning a literal to a variable. In this case, the literal is a hash literal, in which each element is specified by a key/value pair, separated by the symbol =>. Hash literals are delimited by braces. For example:

```
>> kids_ages = {"John" => 38, "Genny" => 36, "Jake" => 22,
"Darcie" => 21}
=> {"Darcie"=>21, "John"=>38, "Genny"=>36, "Jake"=>22}
```

Notice that the order of the hash returned by Ruby is not the same as the hash literal used to create the hash. This is because the actual order of the hash in memory is unpredicatable (at least to the user program).

If the new method is sent to the `Hash` class without a parameter, it creates an empty hash, signified by `{}`. For example:

```
>> my_hash = Hash.new
=> {}
```

An individual value element of a hash can be referenced by "subscripting" the hash name with a key. The same brackets used for array element access are used to specify the subscripting operation. For example:

```
>> kids_ages["Genny"]
=> 36
```

New values are added to a hash by assigning the value of the new element to a reference to the key of the new element, as in the following example:

```
>> kids_ages["Aidan"] = 7;

=> {"Aidan"=>7, "Darcie"=>21, "John"=>38, "Genny"=>36,
"Jake"=>22}
```

An element is removed from a hash with the `delete` method, which takes an element key as a parameter. For example:

```
>> kids_ages.delete("Genny")
=> 36
>> kids_ages
=> {"Aidan"=>7, "Darcie"=>21, "John"=>38, "Jake"=>22}
```

A hash can be set to empty in two ways: First, an empty hash literal can be assigned to the hash. Second, the `clear` method can be used on the hash. These two approaches are illustrated with the following statements:

```
>> hi_temps = {"mon" => 74, "tue" => 78}
=> {"mon"=>74, "tue"=>78}
>> hi_temps = {}
=> {}
>> salaries = {"Fred" => 47400, "Mike" => 45250}
=> {"Fred" => 47400, "Mike" => 45250}
>> salaries.clear
=> {}
```

The `has_key?` predicate method is used to determine whether an element with a specific key is in a hash. For example, assuming the `kids_ages` hash previously defined is still around:

```
>> kids_ages.has_key?("John")
=> true
>> kids_ages.has_key?("Henry")
=> false
```

The keys and values of a hash can be extracted into arrays with the methods `keys` and `values`, respectively as follows:

```
>> kids_ages.keys
=> ["Aidan", "Darcie", "John", "Jake"]
>> kids_ages.values
=> [7, 21, 38, 22]
```

14.7 Methods

Subprograms are central to the usefulness of any programming language. Ruby's subprograms are all methods because it is an object-oriented language. However, Ruby's methods can be defined outside user-defined classes, so both in appearance and in behavior, when defined outside a class, they are like functions. When a method that is defined in a class is called from outside that class, the call must begin with a reference to an object of that class. When a method is called without an object reference, the default object on which it is called is `self`, which is a reference to the current object. Therefore, whenever a method is defined outside a user-defined class, it is called without an object reference. This section describes the basics of Ruby's methods. Classes are introduced in Section 14.8.

14.7.1 Fundamentals

A *method definition* includes the method's header and a sequence of statements, ending with the `end` reserved word, which describes its actions. A *method header* is the reserved word `def`, the method's name, and optionally a parenthesized list of formal parameters. Method names must begin with lowercase letters. If the

method has no parameters, the parentheses are omitted. In fact, the parentheses are optional in all cases, but it is common practice to include them when there are parameters and omit them when there are no parameters. The types of the parameters are not specified in the parameter list, because Ruby variables do not have types. The type of the return object is also not specified in a method definition.

A method that returns an object that is to be used immediately is called in the position of an operand in an expression (or as the whole expression). A method that does not return an object that is to be used can be called by a standalone statement.

A method can specify the value it returns in two ways, explicitly and implicitly. The `return` statement takes an expression as its parameter. The value of the expression is returned when the `return` is executed. A method can have any number of `return` statements, including none. If there are no `return` statements in a method or if execution arrives at the end of the method without encountering a `return`, its implicitly returned object is the value of the last expression evaluated in the method.

The `Time` object is used to obtain various aspects of time from the system clock. The `now` method of `Time` returns the current time and date as a string. This method is used in the following example methods, one with a `return` and one without a `return`:

```
def date_time1
  return Time.now
end
def date_time2
  Time.now
end
```

Consider the following calls to `date_time1` and `date_time2` and the returned values:

```
>> date_time1
=> Thu Jun 07 16:00:06 Mountain Daylight Time 2007
>> date_time2
=> Thu Jun 07 16:00:08 Mountain Daylight Time 2007
```

14.7.2 Local Variables

Local variables are either formal parameters or are variables created in a method. A variable is created in a method by assigning an object to it. The scope of a local variable is from the header of the method to the end of the method. If the name of a local variable conflicts with that of a global variable, the local variable is used. This is the advantage of local variables: When you make up their names, you do not need to be concerned that a global variable with the same name may exist in the program.

The name of a local variable must begin with either a lowercase letter or an underscore (_). Beyond the first character, local variable names can have any number of letters, digits, or underscores.

The lifetime of a variable is the period of time over which it exists and can be referenced. The lifetime of a local variable is from the time it is created until the end of the execution of the method. So, the local variables of a method cannot be used to store data between calls to the method.

14.7.3 Parameters

The parameter values that appear in a call to a method are called *actual parameters*. The parameter names used in the method, which correspond to the actual parameters, are called *formal parameters*. In effect, scalar actual parameters specify the values of objects, not their addresses. So, in Ruby, parameter transmission of scalars is strictly one-way into the method. The values of the scalar actual parameters are available to the method through its formal parameters. The formal parameters that correspond to scalar actual parameters are local variables that are initialized to reference new objects that have the values of the corresponding actual parameters. Whatever a method does to its formal parameters, it has no effect on the actual parameters in the calling program unit. The following example illustrates a method that does not change its parameters:

```
def side3(side1, side2)
  return Math.sqrt(side1 ** 2 + side2 ** 2)
end
```

Now, we consider a method that attempts to change its parameters. The intent of the following method was to interchange its parameters:

```
>> def  swap(x, y)
>>  t = x
>>  x = y
>>  y = t
>> end
=> nil
>> a = 1
>> b = 2
>> swap(a, b)
=> 1
>> a
=> 1
>> b
=> 2
```

So, you see that although `swap` changes its formal parameters, the actual parameters sent to it, `a` and `b`, are unchanged.

Actual parameters that are arrays or hashes are in effect passed by reference, so it is a two-way communication between the calling program unit and the called method. For example, if an array is passed to a method and the method changes the array, those changes are reflected in the corresponding actual parameter in the caller.

Normally, a call to a method must have the same number of actual parameters as the number of formal parameters in the method's definition. A mismatch of these two numbers results in a runtime error. However, a method can be defined to take a variable number of parameters by defining it with a parameter that is preceded by an asterisk (*). Such a parameter is called an *asterisk parameter*. For example:

```
def fun1(*params)
...
end
```

This method can take any number of parameters, including none. The passed actual parameters are placed in the array named `params` (in this example). The asterisk parameter can be preceded by other parameters, in which case only those actual parameters that do not correspond to named formal parameters are placed in the array of parameters. For example, suppose `fun2` is defined as follows:

```
def fun2(sum, list, length, *params)
...
end
```

Now, suppose `fun2` is called with the following:

```
fun2(new_sum, my_list, len, speed, time, alpha)
```

The actual parameters `speed`, `time`, and `alpha` will be passed into the array `params`. Of course, the asterisk parameter must always appear at the end of the list of formal parameters. Any normal parameters that follow an asterisk parameter will always be ignored, because the asterisk parameter receives all remaining actual parameters.

Formal parameters can have default values, which makes their corresponding actual parameters optional. For example, consider the following skeletal method definition:

```
def lister(list, len = 100)
...
end
```

If this method is called with the following, the formal parameter `len` gets the value `50`:

```
lister(my_list, 50)
```

But if it is called with the following, `len` will default to `100`:

```
lister(my_list)
```

Some programming languages, for example Ada and Fortran 95, support keyword parameters. A *keyword parameter* is one in which the actual parameter specifies the name of its associated formal parameter. For example:

```
lister(list => my_list, len => 50)
```

The advantage of keyword parameters is that they eliminate the possibility of making mistakes in the association of actual parameters with formal parameters. This is particularly useful when there are more than a few parameters.

Ruby does not support keyword parameters, but there is a way to achieve the same benefit, using hashes. A hash literal has an appearance that is similar to keyword parameters. For example, if a hash literal is passed as a parameter to the `find` method, it would appear as follows:

```
find(age, {'first' => 'Davy', 'last' => 'Jones'})
```

Whenever such a hash literal is passed as an actual parameter and it follows all normal scalar parameters and precedes all array and block parameters, the braces can be omitted.

Ruby includes a category of objects that appear in no other common programming languages, symbols. Symbols are created by preceding an unquoted string with a colon (`:`). Symbols are not related to variables—`:x` and the variable `x` are unrelated. All symbols are instances of the `Symbol` class. Symbols can be used to specify the keys of elements of hash literals when they are used as parameters. It has become a Ruby idiom, and even a convention in Rails, to use symbols, rather than literal strings, for the keys in hash literals when they are used as parameters. For example:

```
find(age, :first => 'Davy', :last => 'Jones')
```

Following is a method that computes the median of a given array of numbers:

```
# median - a method
#  Parameter: An array of numbers
#  Return value: The median of the parameter array
#
def median(list)

# Sort the array
  list2 = list.sort

# Get the length of the array
  len = list2.length

# Compute the median
  if(len % 2 == 1)  # length is odd
    return list2[len / 2]
```

```
  else                # length is even
    return (list2[len / 2] + list2[len / 2 - 1]) / 2
  end

end  # end of the median method
```

14.8 Classes

Classes in Ruby are like those of other object-oriented programming languages, at least in purpose. A class defines the template for a category of objects, of which any number can be created. An object has state, which is maintained in its collection of instance variables, and behavior, which is defined by its methods. An object can also have constants and a constructor.

14.8.1 The Basics of Classes

The methods and variables of a class are defined in the syntactic container that has the following form:

```
class class_name
...
end
```

Class names, like constant names, must begin with uppercase letters.

Instance variables are used to store the state of an object. They are defined in the class definition and every object of the class gets its own copy of the instance variables. The name of an instance variable must begin with an at sign (`@`), which distinguishes instance variables from other variables.

A class can have a constructor, which in Ruby is a method with the name `initialize`, which is used to initialize instance variables to values. A constructor can take any number of parameters, which are treated as local variables, and therefore their names begin with lowercase letters or underscores. The parameters are given after the call to `new`.

Following is an example of a class. This class, named `Stack2_class`, defines a stack-like data structure implemented in an array. The difference between this structure and a stack is that both the top and second from the top elements are accessible. The second from the top element is fetched with the `top2` method.

```
# Stack2_class.rb - a class to implement a stack-like
#                   structure in an array
class Stack2_class

# Constructor - parameter is the size of the stack - default is 100
```

```
  def initialize(len = 100)
    @stack_ref = Array.new(len)
    @max_len = len
    @top_index = -1
  end

# push method
  def push(number)
    if @top_index == @max_len
      puts "Error in push - stack is full"
    else
      @top_index += 1
      @stack_ref[@top_index] = number
    end
  end

# pop method
  def pop()
    if @top_index == -1
      puts "Error in pop - stack is empty"
    else
      @top_index -= 1
    end
  end

# top method
  def top()
    if @top_index > -1
      return @stack_ref[@top_index]
    else
      puts "Error in top - no elements"
    end
  end

# top2 method
  def top2
    if @top_index > 0
      return @stack_ref[@top_index - 1]
    else
      puts "Error in top2 - there are not 2 elements"
    end
  end

# empty method
```

```
  def empty()
    @topIndex == -1
  end

end
```

Following is simple code to illustrate the use of the `Stack2_class` class:

```
# Test code for Stack2_class
  mystack = Stack2_class.new(50)
  mystack.push(42)
  mystack.push(29)
  puts "Top element is (should be 29): #{mystack.top}"
  puts "Second from the top is (should be 42): #{mystack.top2}"
  mystack.pop
  mystack.pop
  mystack.pop  # Produces an error message - empty stack
```

Classes in Ruby are dynamic in the sense that members can be added at any time. This is done by simply including additional class definitions that specify the new members. Methods can also be removed from a class. This is done by providing another class definition in which the method to be removed is sent to the method `remove_method` as a parameter. The dynamic classes of Ruby are another example of a language designer trading readability (and as a consequence, reliability) for flexibility. Allowing dynamic changes to classes clearly adds flexibility to the language, but harms readability. To determine the current definition of a class, one must find all of its definitions in the program and consider all of them.

14.8.2 Access Control

In a clear departure from the other common programming languages, access control in Ruby is different for access to data than it is for access to methods. All instance data has private access by default, and it cannot be changed. If external access to an instance variable is required, access methods must be defined. For example, consider the following skeletal class definition:

```
class My_class

# A constructor
  def initialize
    @one = 1
    @two = 2
  end
```

```
# A getter for @one
  def one
    @one
  end

# A setter for @one
  def one=(my_one)
    @one = my_one
  end

end  # of class My_class
```

The equal sign (=) attached to the name of the setter method means that the method is assignable. So, all setter methods have equal signs attached to their names. The body of the `one` method illustrates the Ruby design of methods returning the value of the last expression evaluated when there is no return statement. In this case, the value of `@one` is returned.

Because getter and setter methods are so frequently needed, Ruby provides shortcuts for both. If one wants a class to have getter methods for two instance variables, `@one` and `@two`, those getters can be specified with the single statement in the class as follows:

```
attr_reader :one, :two
```

`attr_reader` is actually a method call, using the symbols `:one` and `:two` as the actual parameters.

The function that similarly creates setters is called `attr_writer`. This function has the same parameter profile as `attr_reader`.

The functions for creating getter and setter methods are so named because they provide the protocol for objects of the class, which in Ruby are called *attributes*. So, the attributes of a class is the data interface (the public data) to objects of the class.

The three levels of access control for methods are defined as follows. Public access means the method can be called by any code. Protected access means that only objects of the defining class and its subclasses may call the method. Private access means that the method cannot be called with an explicit receiver object. Because the default receiver object is `self`, a private method can only be called in the context of the current object. So, no code can ever call the private methods of another object. Note that private access in Ruby is quite different from private access in other programming languages such as C++, Java, and C#.

Access control for methods in Ruby is dynamic, so access violations are detected only during execution. The default method access is public, but it can also be protected or private. There are two ways to specify the access control, both of which use functions with the same names as the access levels, `private`, `protected`, and `public`. One way is to call the appropriate function without parameters. This resets the default access for all following defined methods in the class until a call to a different access control method appears. For example:

```
class My_class
  def meth1
  ...
  end
  ...
private
  def meth7
  ...
  end
  ...
protected
  def meth11
  ...
  end
  ...
end  # of class My_class
```

The alternative is to call the access control functions with the names of the specific methods as parameters. For example, the following is semantically equivalent to the previous class definition:

```
class My_class
  def meth1
  ...
  end
  ...
  def meth7
  ...
  end
  ...
  def meth11
  ...
  end
  ...
  private :meth7, ...
  protected :meth11, ...
  end  # of class My_class
```

The default access control for constructors is private. Class variables are private to the class and its instances. That privacy cannot be changed. Also, unlike global and instance variables, class variables must be initialized before they are used.

14.8.3 Inheritance

Subclasses are defined in Ruby using the less-than symbol (<).

```
class My_Subclass < Base_class
```

One distinct thing about the method access controls of Ruby is that they can be changed in a subclass, simply by calling the access control functions. This means that two subclasses of a base class can be defined so that objects of one of the subclasses can access a method defined in the base class, but objects of the other subclass cannot. Also, this allows one to change the access of a publically accessible method in the base class to a privately accessible method in the subclass. Such a subclass obviously cannot be a subtype.

Ruby modules provide a naming encapsulation that is often used to define libraries of methods. Perhaps the most interesting aspect of modules, however, is that their methods can be accessed directly from classes. Access to the module in a class is specified with an `include` statement, such as the following:

```
include Math
```

The effect of including a module is that the class gains a pointer to the module and effectively inherits the functions defined in the module. In fact, when a module is included in a class, the module becomes a proxy superclass of the class. Such a module is called a *mixin*, because its functions get mixed into the methods defined in the class. Mixins provide a way to include the functionality of a module in any class that needs it. And, of course, the class still has a normal superclass from which it inherits members. So, mixins provide the benefits of multiple inheritance, without the naming collisions that could occur if modules did not require module names on their functions.

14.9 Code Blocks and Iterators

A block is a segment of code, delimited by either braces or the `do` and `end` reserved words. By itself, a block does nothing other than visually set off its code segment. However, blocks can be used with specially written methods to create many useful constructs, including simple iterators for arrays and hashes. This construct consists of a method call followed by a block. In the following paragraphs, a few of the built-in iterator methods that are designed to use blocks are discussed.

The `times` iterator method provides a way to build simple counting loops. Typically, `times` is sent to a number, which repeats the attached block that number of times. Consider the following example:

```
>> 4.times {puts "Hey!"}
Hey!
Hey!
Hey!
Hey!
=> 4
```

The `times` method repeatedly executes the block. This is a different approach to control of a subprogram, of which the block is clearly a form.

The most commonly used iterator is `each`, which is often used to go through arrays and apply a block to each element. For this, it is convenient to allow blocks to have parameters. Blocks *can* have parameters, which appear at the beginning of the block, delimited by vertical bars (|). The following example, which uses a block parameter, illustrates the use of `each`:

```
>> list = [2, 4, 6, 8]
=> [2, 4, 6, 8]
>> list.each {|value| puts value}
2
4
6
8
=> [2, 4, 6, 8]
```

The `each` iterator works equally well on array literals, as in the following:

```
>> ["Joe", "Jo", "Joanne"].each {|name| puts name}
Joe
Jo
Joanne
=> ["Joe", "Jo", "Joanne"]
```

The `upto` iterator method is used like `times`, except that the last value of the counter is given as a parameter. For example:

```
>> 5.upto(8) {|value| puts value}
5
6
7
8
=> 5
```

The `step` iterator method takes a terminal value and a step size as parameters and generates the values from that of the object to which it is sent and the terminal value. For example:

```
>> 0.step(6, 2) {|value| puts value}
0
2
4
6
=> 0
```

The `collect` iterator method takes the elements from an array, one at a time, like `each`, and puts the values generated by the given block into a new array. For example:

```
>> list = [5, 10, 15, 20]
=> [5, 10, 15, 20]
```

```
>> list.collect {|value| value = value - 5}
=> [0, 5, 10, 15]
>> list
=> [5, 10, 15, 20]
>> list.collect! {|value| value = value - 5}
=> [0, 5, 10, 15]
>> list
=> [0, 5, 10, 15]
```

As can be seen from this example, the mutator version of `collect` is probably more often useful than the non-mutator version, which does not save its result.

Now we consider user-defined methods and code blocks. There must be some statement in the method that "calls" the block. This statement is `yield`. The `yield` statement is similar to a method call, except that there is no receiver object and the call is a request to execute the code block attached to the method call, rather than a call to a method. If the code block has parameters, they are specified in parentheses on the `yield` statement. The value returned by a code block is that of the last expression evaluated in the block. A method can include any number of `yield` statements, so it can cause the block to be "called" any number of times. It is this process that is used to implement the iterators illustrated earlier in this section.

When a code block is used in a call to a method, part of the effect of the call is provided by the code in the method and part is provided by the code block. This allows a method to have differect effects on different calls, with the different effects provided by the code block atttached to the call. Consider the following example:

```
>> def get_name
>>  puts "Your name:"
>>  name = gets
>>  yield(name)
>> end
=> nil
>> get_name {|name| puts "Hello, " + name}
Your name:
Freddie
Hello, Freddie
=> nil
```

14.10 Pattern Matching

Regular expressions in JavaScript were discussed in Chapter 4, "The Basics of JavaScript." Because the regular expressions of both JavaScript and Ruby are based directly on those of Perl, readers who are not familiar with regular expressions are referred to Sections 4.12.1 to 4.12.3. The pattern-matching operations of Ruby are different from those of Perl and JavaScript, so they are discussed here.

14.10.1 The Basics of Pattern Matching

In Ruby, the pattern-matching operation is specified with the matching operators, `=~`, which is for positive matches, and `!~`, which is for negative matches. Patterns are placed between slashes (`/`). For example, in the following the right operand pattern is matched against the left operand string:

```
>> street = "Hammel"
=> "Hammel"
>> street =~ /mm/
=> 2
```

The result of evaluating a pattern-matching expression is the position in the string where the pattern matched.

The `split` method is frequently used in string processing. It uses its parameter, which is a pattern, to determine how to split the string object to which it is sent into substrings. For example, we could have the following:

```
>> str = "Jake used to be a small child, but now is not."
=> "Jake used to be a small child, but now is not."
>> words = str.split(/[ .,]\s*/)
=> ["Jake", "used", "to", "be", "a", "small", "child",
"but", "now", "is", "not"]
```

This statement puts the words from `str` into the `words` array, where the words in `str` are defined to be terminated with either a space, a period, or a comma, any of which could be followed by more whitespace characters.

The following sample program illustrates a simple use of pattern matching and hashes. The program reads lines of text in which the words are separated by whitespace and some common kinds of punctuation such as commas, periods, semicolons, and so forth. The objective of the program is to produce a frequency table of the words found in the input. A hash is an ideal way to build the word-frequency table. The keys can be the words, and the values can be the number of times they have appeared. The `split` method provides a convenient way to split each line of the input file into its words. For each word, the program uses `has_key?` on the hash to determine whether the word has occurred before. If so, its count is incremented; if not, the word is entered into the hash with a count of `1`.

```
# word_table.rb
#  Input: Text from the keyboard. All words in the input are
#     separated by whitespace or punctuation, possibly followed
#     by whitespace, where the punctuation can be a comma, a
#     semicolon, a question mark, an exclamation point, a period,
#     or a colon.
# Output: A list of all unique words in the input, in alphabetical
#          order, along with their frequencies of occurrence
```

```
freq = Hash.new
line_words = Array.new

# Main loop to get and process lines of input text
while line = gets

# Split the line into words
    line_words = line.chomp.split( /[ \.,;:!\?]\s*/)

# Loop to count the words (either increment or initialize to 1)
    for word in line_words
        if freq.has_key?(word) then
            freq[word] = freq[word] + 1
        else
            freq[word] = 1
        end
    end
end
# Display the words and their frequencies
puts "\n Word \t\t Frequency \n\n"
for word in freq.keys.sort
    puts " #{word} \t\t #{freq[word]}"
end
```

Notice that the two normally special characters, `.` (period) and `?` (question mark), are not backslashed in the pattern for `split` in this program. This is because, as mentioned previously, the normally special characters for patterns (metacharacters) are not special in character classes.

14.10.2 Remembering Matches

The part of the string that matched a part of the pattern can be saved in an implicit variable for later use. The part of the pattern whose match you want to save is placed in parentheses. The substring that matched the first parenthesized part of the pattern is saved in `$1`, the second in `$2`, and so forth. As an example, consider the following:

```
>> str = "4 July 1776"
=> "4 July 1776"
>> str =~ /(\d+) (\w+) (\d+)/
=> 0
>> puts "#{$2} #{$1}, #{$3}"
=> July 4, 1776
```

In some situations, it is convenient to be able to reference the parts of the string that preceded the match, the part that matched, or the part that followed the match. These three strings are available after a match through the implicit variables `$``, `$&`, and `$'`, respectively.

14.10.3 Substitutions

Sometimes the substring of a string that matched a pattern must be replaced by another string. Ruby's `String` class has four methods designed to do exactly that. The most basic of these, the substitute method, `sub`, takes two parameters, a pattern and a string (or expression that evaluates to a string value). `sub` matches the pattern against the string object to which it is sent. If `sub` finds a match, the matched substring is replaced by its second parameter. Consider the following examples:

```
>> str = "The old car is great, but old"
=> "The old car is great, but old"
>> str.sub(/old/, "new")
=> "The new car is great, but old"
```

The `gsub` method is similar to `sub`, but finds all substring matches and replaces all of them with its second parameter. For example:

```
>> str = "The old car is great, but old"
=> "The old car is great, but old"
>> str.gsub(/old/, "new")
=> "The new car is great, but new"
>> str
=> "The old car is great, but old"
```

Notice from the last line that `gsub` does not alter the string object on which it is called. The same is true for `sub`. However, `sub` and `gsub` have mutator versions, named `sub!` and `gsub!`. For example:

```
>> str = "The old car is great, but old"
=> "The old car is great, but old"
>> str.gsub!(/old/, "new")
=> "The new car is great, but new"
>> str
=> "The new car is great, but new"
```

The `i` modifier, which tells the pattern matcher to ignore the case of letters, can also be used with the substitute method by attaching it to the right end of the pattern, as shown in the following code:

```
>> str = "Is it Rose, rose, or ROSE?"
=> "Is it Rose, rose, or ROSE?"
>> str.gsub(/rose/i, "rose")
=> "Is it rose, rose, or rose?"
```

Summary

Ruby is a pure object-oriented language that is interpreted. Perhaps the primary motivation for its popularity is its use in the Rails framework for building Web applications.

Ruby has three categories of data types: scalars, arrays, and hashes. The scalar classes are `Float`, `Fixnum`, `Bignum`, and `String`. Ruby's arithmetic expressions and assignment statements are like those of other common languages. All Ruby variables are references to objects. It has no primitive types such as C++, Java, and C#. Although expressions appear in the same form as in other languages, underneath they are all executed by message passing. The `String` class has a large number of methods.

Ruby includes the usual collection of control statements, including two different multiple-selection statements. Arrays in Ruby are different from the more conventional languages in that they can store any objects and they have dynamic length. The `Array` class provides a large collection of methods, including those for implementing stacks and queues in arrays. Ruby's hashes are similar to those of Perl.

Methods can be defined in classes, but also outside classes, in which case they are much like functions. Asterisk formal parameters provide the means of supporting a variable number of parameters. Classes are dynamic, in the sense that methods and variables can be added or deleted at any time. Access control is provided by calling the `public`, `private`, and `protected` methods. Ruby includes an implicit way to provide getters and setters.

One unique feature of Ruby is its code blocks and iterators. The `each` and `find` iterators are frequently used to deal with arrays. Ruby's pattern matching operations use the same regular expressions as JavaScript and Perl.

Review Questions

14.1 What is one of the most common uses of Ruby?

14.2 What are the two integer classes of Ruby?

14.3 What is the length limit of a `Bignum` object?

14.4 What is the difference between the two kinds of string literals?

14.5 What numeric operators in C and Java are missing in Ruby?

14.6 What does the `String` method `replace` do?

14.7 What is the difference between the `downcase` and `downcase!` methods?

14.8 What values of a variable are considered true?

14.9 What are the syntactic differences between the JavaScript `if` statement and that of Ruby?

14.10 In what two ways can an `Array` object be created?

14.11 Describe what the `for-in` statement does.

14.12 Describe how the concatenation operator for arrays works.

14.13 What does the `include?` method do?

14.14 What is the form of a hash literal?

14.15 Do method headers require parentheses?

14.16 What is an asterisk parameter?

14.17 What is the form of an instance variable's name?

14.18 When are access control violations for methods detected?

14.19 Explain what the `each` method does.

Exercises

14.1 Write, test, and debug (if necessary) a Ruby program for the following specification.

Input: Three numbers, `a`, `b`, and `c`, each on its own line, from the keyboard.

Output: The value of the expression `10ab-((c-1)/17.44)`.

14.2 Write, test, and debug (if necessary) a Ruby program for the following specification.

Input: A list of numbers from the keyboard.

Output: The second smallest number in the list, along with its position in the list, with 1 being the position of the first number.

14.3 Write, test, and debug (if necessary) a Ruby program for the following specification.

Input: Three names, on separate lines, from the keyboard.

Output: The input names in alphabetical order, without using arrays.

14.4 Write, test, and debug (if necessary) a Ruby program for the following specification.

Input: A list of lines of text from the keyboard.

Output: Every input line that has more than 10 characters (not counting the newline) but fewer than 20 characters (not counting the newline) that contains the string "ed".

14.5 Write, test, and debug (if necessary) a Ruby program for the following specification.

Input: A list of numbers from the keyboard.

Output: Two lists of numbers, one with input numbers that are greater than zero and one with those that are less than zero (ignore the zero-valued numbers). You must first build two arrays with the required output numbers before you display any of them.

14.6 Write, test, and debug (if necessary) a Ruby program for the following specification.

Input: A list of numbers from the keyboard.

Output: The median of the input numbers.

CHAPTER

15

Introduction to Rails

15.1 Overview of Rails
15.2 Document Requests
15.3 Processing Forms
15.4 Rails Applications with Databases
15.5 Layouts
Summary • Review Questions • Exercises

As stated in Chapter 14, our primary interest in Ruby in this book is its use with the Web software development framework, Rails. This chapter introduces Rails. Rails is a complex system with a large array of powerful capabilities, and this is but one chapter of a book. Therefore, only a rudimentary introduction to a few of the most straightforward uses of Rails will be discussed. The chapter begins with an overview of Rails, including its approach to combining the MVC model of applications with relational databases. The remainder of the chapter is a discussion of Rails through a seqence of example applications, beginning with the simplest of applications, Hello World. This is followed by an application to serve a simple dynamic document. Next, an example of form handling is discussed. After that, a complete application that accesses a database is described. Included is the process of creating the database and setting it up for use with a Rails application.

15.1 Overview of Rails

Rails is a development framework for Web-based applications, including those that access databases. A framework is a system in which much of the more-or-less standard software parts are furnished by the framework, so they do not need to be written by the applications developer. Rails, because of its intimate connection with Ruby, is often called Ruby on Rails, or simply RoR. Rails was developed by David Heinemeier Hansson in the early 2000s and was released to the public in July 2004. Since then, it has rapidly gained widespread interest and usage.

Rails, like some other Web development frameworks, such as Tapestry and Struts, is based on the Model-View-Controller (MVC) architecture for applications. MVC was developed by Trygve Reenskaug, a Norwegian, in 1978–1979 while he was a visiting scientist at XeroxPARC working in the Smalltalk group. The original intent of MVC was to model graphical user interfaces, which were then being developed for Smalltalk. The MVC architecture clearly separates applications, both logically and physically, into three parts. The *model* is not only the data, but any enforced constraints on the data. For example, if a part of the data is the age of people, the model might ensure that no age value outside the usual range of possible human ages can be entered into the data storage. The *view* is the part of an application that prepares and presents results to the user. The *controller*, true to its namesake, controls the application. In addition, the controller performs any required computations. In an MVC Web application, a browser submits requests to the controller, which consults the model (which in turn consults its database) and reports results back to the controller and indirectly to the view. The controller then instructs the view to produce a result document that is then transmitted to the client for display. The intent of MVC is to reduce the coupling among the three parts of an application, making the application easier to write and maintain.

A significant and characterizing part of Rails is its approach to connecting object-oriented software with a relational database. These two are not particularly amenable to marriage, so this is not a natural connection. For this, Rails uses an object-relational mapping (ORM) approach. Each relational database table is implicitly mapped to a class. For example, if the database has a table named `cars`, the Rails application program that uses `cars` will have a class named `Cars`. Rows of the `cars` table will have corresponding objects of the `Cars` class. The `Cars` class will have methods to get and set the various state (instance) variables of objects of the class. In summary, an ORM maps tables to classes, rows to objects, and columns to the fields of the objects. Furthermore, the `Cars` class will have class methods for performing table-level operations, such as finding an object of a certain attribute. The key aspect of the ORM in Rails is that it is implicit. The classes, objects, methods, and instance variables that represent a database in Ruby are automatically built by Rails. The ORM of Rails is named *ActiveRecord*.

The view and controller parts of MVC is supported with the *ActionPack* component of Rails. Although they are packaged in the same component, the code support for the view and controller are cleanly separated. The view docu-

ments are generated with three different kinds of templates, corresponding to the three categories of view documents, XHTML, XML, and JavaScript scripts that dynamically generate documents on the browser.

The controller part of MVC controls the interactions between the data model, the user, and the view. Most of the code to support these interactions are provided by Rails. So, most of what the developer writes for the controller is the logic of the specific application.

Rails can be and often is used in conjunction with Ajax. Rails uses a JavaScript framework named Prototype to support Ajax and interactions with the JavaScript model of the document being displayed by the browser. Rails also provides other support for developing Ajax, including producing visual effects.

A Rails application is a program that provides a response when a client browser connects to a Rails-driven Web site. Building a Rails application, because Rails uses an MVC architecture, consists of designing and building the three parts of an MVC system. The developer must design and build a model of the application's domain. This model must include the design of a database that represents the model. The database itself is constructed with tools provided by Rails. Some of it is implicitly built as a side effect of building the parts of the model. For example, if the application is an online bookstore, the parts are the inventory and a catalog of all that can be ordered through the store, among other things. Next, the developer must design and build the actions that can happen in the operation of the bookstore, such as inquiries, purchases, orders, and billing. Finally, the developer must design and build the publicly accessible views of the application domain.

15.2 Document Requests

Rails is a software development framework. A software development framework is often constructed as a library of components that provide commonly needed services to an application in a particular application area. The application area for Rails is Web applications that use relational databases, which are discussed in Chapter 13, "Database Access Through the Web."

Before one can interact with Rails, the system must be downloaded and installed on one's computer system. For Windows users, the simplest way to do this is to download a complete software system named InstantRails from `http://instantrails.rubyforge.org/wiki/wiki.pl`. InstantRails, which was developed by Curt Hibbs, includes Ruby, Rails, MySQL, Apache, and everything else necessary to use these technologies together. Installing InstantRails is quick and easy.

InstantRails is a self-contained system. It does not reside in the global Windows environment, so interactions with it cannot be done through a Windows command window. Yes, it is run from a command prompt, not a GUI, but it uses its own command window.[1]

1. Of course, InstantRails could be configured to run through a Windows command window, but there is no benefit of doing that.

For UNIX systems, Rails is available at the following address:

```
http://www.RubyonRails.org/down
```

15.2.1 Static Documents—Hello World in Rails

This section describes how to build a Hello World application in Rails. The purpose of such an exercise is to demonstrate the structure of the simplest possible Rails application, showing what files must be created and where they must reside in the directory structure. InstantRails itself runs in the following directory:

```
C:\myrails\InstantRails-1.7-win\InstantRails
```

where the first directory's name, `myrails` in this example, is chosen by the person who installed InstantRails. In this directory there is an application file named InstantRails. Running this application opens a small window, as shown in Figure 15.1.

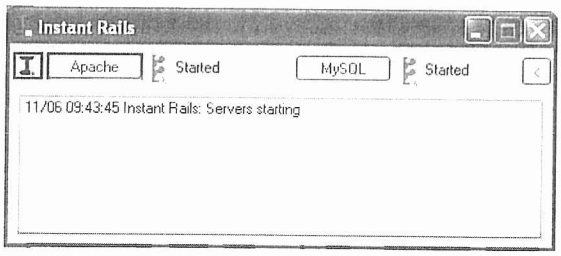


Figure 15.1 The InstantRails application window

A click on the black "I" on the left border of this window produces a small menu. Click the `Rails Application` entry in this menu, which opens another menu. Clicking on the `Open Ruby Console Window` entry in this menu opens a DOS command window in the following directory:

```
C:\myrails\InstantRails-1.7-win\InstantRails\rails_aps
```

It is at this command line that Rails commands can be given. They cannot be given in a normal DOS command window.

To this base directory, users usually add a new subdirectory for all their Rails applications. We named ours `exercises`. In the new `exercises` directory, the new application `rails1` is created with the following command:

```
>rails rails1
```

Rails responds by creating more than 40 files in more than 15 directories. This is part of the framework to support a Rails application. Directly under the specific application directory, in this case `rails1`, 12 subdirectories are created,

the most interesting of which at this point is app. The app directory has four subdirectories, models, views, and controllers, which correspond directly to the MVC architecture of a Rails application, and helpers. The helpers subdirectory contains Rails-provided methods that aid in constructing applications. The user code to support the application will reside in either models, views, or controllers or subdirectories of them.

One of the directories created by the rails command is script, which has several important Ruby scripts that perform services. One of these, generate, is used to create part of an application controller. This script creates two Ruby controller methods, and also a subdirectory of the views directory where views code will be stored. For our application, we pass two parameters to generate, the first of which is controller, which indicates that we want the controller class built. The second parameter is the name we chose for the controller. An important part of how Rails works is its focused use of names. Our first example of this is the name of the controller. This name will also be part of the file name of the controller class and part of the name of the controller class. In addition, it will be the name of the subdirectory of the views directory and a part of the URL of the application. For our example, the following command is given in the rails1 directory to create the controller:

```
>ruby script/generate controller say
```

With this command we have chosen the name say for the controller for our application.The response produced by the execution of this command follows:

```
exists  app/controllers/
exists  app/helpers/
create  app/views/say
exists  test/functional/
create  app/controllers/say_controller.rb
create  test/functional/say_controller_test.rb
create  app/helpers/say_helper.rb
```

The exists lines above indicate files and directories that are verified to already exist. The create lines show the newly created directories and files. There are now two Ruby classes in the controllers directory, application.rb and say_controller.rb, where the say_controller.rb class is a subclass of application.rb, which provides the default behavior defined in the parent class. The say_controller.rb is the specific controller for the rails1 application. Following is a listing of say_controller.rb:

```
class SayController < ApplicationController
end
```

Note the use of say in both the name of the controller file and the controller class. The directory structure built on the exercises directory is shown in Figure 15.2.

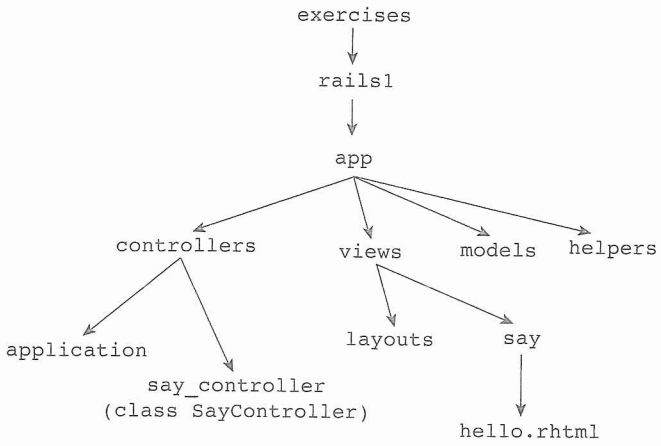


Figure 15.2 Directory structure for the `rails1` application

`SayController` is an empty class, other than what it inherits from `application.rb`. The controller produces, at least indirectly, the response to requests, so a method must be added to it. The method does not need to actually do anything, other than indicate a document that will be the response. The mere existence of the method specifies by its name the response document. So, the action will be nothing more than an empty method definition, whose name will be the same as that of the response document in the `say` subdirectory of `views`. With the empty method, which is called an *action method*, the controller now has the following appearance:

```
class SayController < ApplicationController
  def hello
  end
end
```

Browser requests to a Rails application have an abbreviated form. The first part of the requested URL is the address of the server. Because we run our examples on the same machine on which the application resides, the server is always `localhost`. The last part of the URL is the name of the controller and the action method name. So, for the `rails1` example, the request URL is as follows:

```
http://localhost/say/hello
```

(Now it should be obvious why the base document is named `say`.) In some cases, a port number must be included in the URL. We will get back to that shortly.

Next, we need to build the view file, which will be a simple XHTML file to produce the greeting. The view document is often called a *template*. The following is the template for the `rails1` application:

```
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- hello.rhtml - the template for Hello World
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Hello, Rails! </title>
  </head>
  <body>
    <h1> Hello from Rails! </h1>
  </body>
</html>
```

The extension on this file is `.rhtml` for the same reason XHTML documents that include PHP scripts use the extension `.php`. Templates can include Ruby code, which is interpreted by a Ruby interpreter named ERb, for *E*mbedded *Ruby*, before the template is returned to the requesting browser.

The template file for our application resides in the `say` subdirectory of the `views` subdirectory of the `app` subdirectory of the `rails1` directory.

Before the application can be tested, a Rails Web server must be started. A server is started with the `server` script from the `script` directory. Within Rails there are three different servers available. The default server is `Mongrel`, but `Apache` and `WEBrick` are also available in Rails. Because it is the default Rails server, `Mongrel` can be started with the following command at the application prompt:

```
>ruby script/server
```

The default port is `3000`. If a different port must be used, because `3000` is already being used by some other program on the system, the port number is given as a parameter to the `server` script. Assuming `3000` is the port to be used, the complete URL of our application is as follows:

```
http://localhost/say/hello
```

If port `3005` is used instead of `3000`, the URL would be as follows:

```
http://localhost:3005/say/hello
```

Note that the server is started by a command in the subdirectory of the particular application. This implies that no other application can be served by this server.

If you are running Windows and have the IIS server running, you will need to stop it before starting the Rails-supplied server. Also, after stopping IIS, you may need to restart the default Web site.

Figure 15.3 shows the output of the `rails1` application when it is addressed by a browser.

Figure 15.3 The reponse from `rails1`

The following summarizes how Rails reacts to a request for a static document: First, the name of the controller is extracted from the URL (it follows the server name). Next, an instance of the controller class (found in the `app/controllers` subdirectory), in our example, `SayController`, is created. The name of the action is then extracted from the URL, in our example, `hello`. Next, Rails searches for a template with the same name as the action method in the subdirectory with the same name as the controller in the `app/views` directory. The template file is then given to ERb to interpret any Ruby code that is embedded in the template. In the case of `hello.rhtml`, there is no embedded Ruby code, so this step has no affect. Finally, the template file is returned to the requesting browser, which displays it. The activities of Rails in response to a simple request are shown in Figure 15.4.

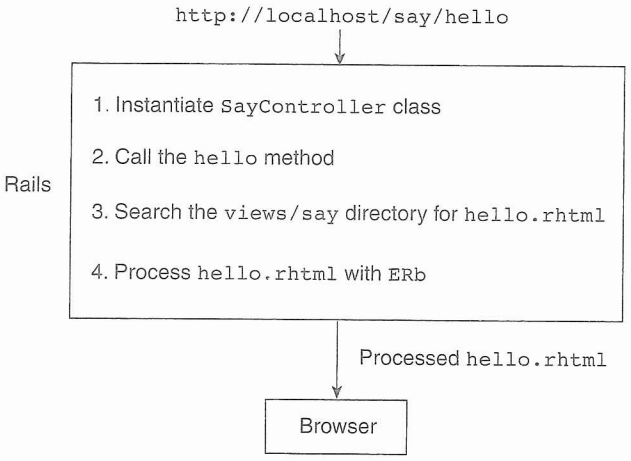


Figure 15.4 Rails actions for a simple request

15.2.2 Dynamic Documents

Rails offers three different approaches to producing dynamic documents. In this chapter, only one of these is discussed, which is to embed Ruby code in a tem-

plate file. This is similar to some other approaches we have discussed, in particular PHP, ASP.NET, and JSP.

As an example of a dynamic document, we construct a new application that gives a greeting, but also displays the current date and time, including the number of seconds since midnight (just so some computation would be included). This application is named `rails2` and the controller is named `time`. This application will illustrate how Ruby code that is embedded in a template file can accesse instance variables that are created and assigned values in the action method of the controller.

Ruby code is embedded in a template file by placing it between the `<%` and `%>` markers. If the Ruby code produces a result and the result is to be inserted into the template document, an equal sign (=) is attached to the opening marker. For example:

```
<p> The number of seconds in a day is: <%= 60 * 60 * 24 %>
</p>
```

After interpretation, this is as follows:

```
<p> The number of seconds in a day is: 86400 </p>
```

The date can be obtained by calling Ruby's `Time.now` method. This method returns the current day of the week, month, day of the month, time, time zone,[2] and year, as a string. So, we can put the date in the response template with:

```
<p> It is now <%= Time.now %> </p>
```

The value returned by `Time.now` can be parsed with the methods of the `Time` class. For example, the `hour` method returns the hour of the day, the `min` method returns the minutes of the hour, and the `sec` method returns the seconds of the minute. These methods can be used to compute the number of seconds since midnight. Putting these together results in the following template file:

```
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- timer.rhtml - Response document for rails2 -
     Hello World + time
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> rails2 example </title>
```

2. The time zone is represented as the number of hours from Coordinated Universal Time (UTC). For Mountain Daylight Time, this is –0600 for six hours after UTC.

```
  </head>
  <body>
    <h2> Hello World! </h2>
    <p>
      It is now <%= t = Time.now %> <br />
      Number of seconds since midnight:
      <%= t.hour * 3600 + t.min * 60 + t.sec %>
    </p>
  </body>
</html>
```

In this case, the template file resides in the `time` subdirectory (`time` is the name of the controller of this application) of the `views` subdirectory of the `app` subdirectory of the `rails2` directory.

Figure 15.5 shows the display of the `rails2` application.

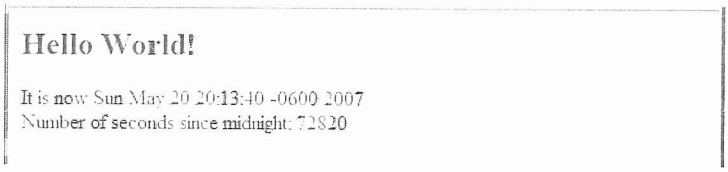


Figure 15.5 The output of `rails2 (time/timer)`

It would be better to place the Ruby code of `rails2` in the controller, because that would separate the program code from the markup. In this case, it does not amount to much code, but we will show it as a new application named `rails3`, with the controller named `time2` with an action method named `timer2`, just to illustrate how it would appear. The controller class would be as follows:

```
class Time2Controller < ApplicationController
  def timer2
    @t = Time.now
    @tsec = @t.hour * 3600 + @t.min * 60 + @t.sec
  end
end
```

The response template now needs to be able to access the instance variables in the `Time2Controller` class. Rails makes this trivial, for all instance variables in the controller class are visible to the template. The template file for `rails3` is shown in the following program.

```
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- timer2.rhtml - Response document for rails3 -
     Hello World + time
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> rails3 example </title>
  </head>
  <body>
    <h2> Hello World! </h2>
    <p>
      It is now <%= @t %> <br />
      Number of seconds since midnight:
      <%= @tsec %>
    </p>
  </body>
</html>
```

15.3 Processing Forms

The popcorn order form used in Chapter 2 (Section 2.9) and Chapter 11 (Section 11.10) is used here to illustrate form handling with Rails. Recall that the popcorn application presents an order form for several different popcorn products and produces a bill to the user when an order form is submitted. In Chapter 11, this form is handled with a PHP script. Here we handle it with Rails.

15.3.1 Setting Up the Application

As with all Rails applications, we begin by creating an application subdirectory under the `rails_apps` directory (`rails_apps` is a subdirectory of `InstantRails`). In this case, the subdirectory is named `popcorn`. We then switch to the new directory and generate a controller named `home` with the following command:

```
>ruby script/generate controller home
```

We then open that controller, whose file is named `home_controller.rb`. (It resides in the `controllers` directory.) We add an empty action method named `the_form` to this class.

The next step in developing this application is to add the `the_form.rhmtl` template to the `home` subdirectory of the `views` directory of our application. The contents of `the_form.rhtml` is exactly the same as the popcorn HTML files in Chapters 2 and 11, except that the opening form tag appears as follows:

```
<form action = "result"  method = "post">
```

This specifies that the name of the action method in the application's controller, as well as the template for the result of submitting the form, is `result`. Notice that this tag specifies the POST HTTP method. Rails requires that POST be used.

Now, we point our browser to the following:

```
http://localhost/home/the_form
```

The resulting display is shown in Figure 15.6.

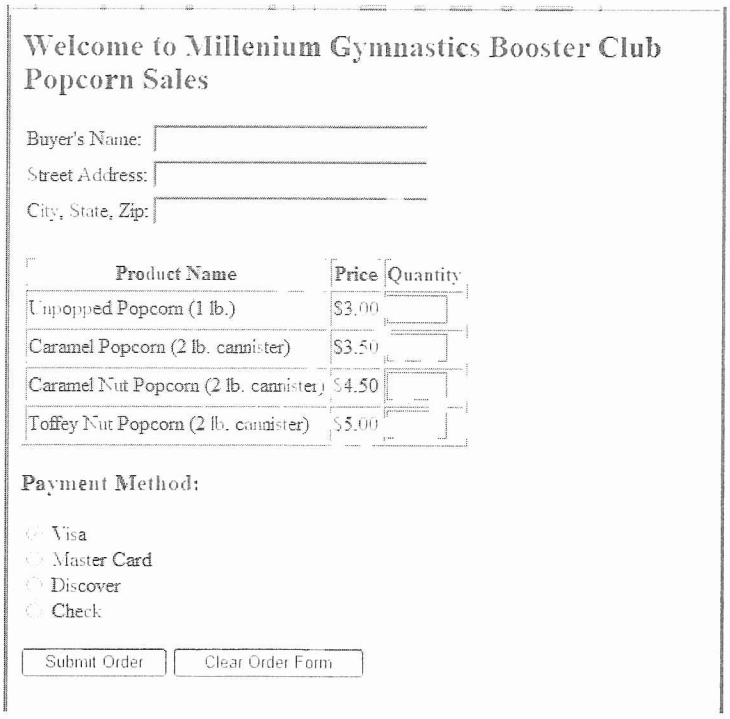


Figure 15.6 The popcorn application initial display

15.3.2 The Controller and the View

The next step of the construction of the application is to build the action method in `home_controller.rb` to process the form data when the form is submitted. In the initial template file, `the_form.rhtml`, this method is named `result` in the `action` attribute of the form tag.The `result` method has two tasks, the first of which is to fetch the form data. This data is used to display back to the customer and to compute the results. The form data is made available to the controller class through the Rails-defined object, `params`. `params` is a hash-like object that contains all of the form data (as well as some other things). It is hash-like because it is a hash that can be indexed with either symbols or actual keys (a hash can be indexed only with keys). The common Rails convention is to index `params` with symbols. For example, to fetch the value of the form element whose name is `phone`, we would use the following:

```
@phone = params[:phone]
```

Of course, all form data is in string form. However, some of the values are integer numeric quantities, so they must be converted to integers with the `to_i` method of `String`. The form of the statements to fetch the form data is illustrated by the following statement:

```
@unpop = params[:unpop].to_i
```

Notice that the instance variable has the same name as the form element. In this case, the value is a quantity, which is converted to an integer. The other quantities on the form are those for the form elements named `caramel`, `caramelnut`, and `toffeynut`. In addition, the string values for `name`, `street`, `city`, and `payment` must be fetched.

The computations for the application are for the cost of each variety of popcorn, the total number of items ordered, and the total cost of the order. The unit prices are as follows:

unpopped corn	$3.00
caramel corn	$3.50
caramel nut corn	$4.50
toffey nut corn	$5.00

The computations are relatively simple. The only complication is that people prefer that when money amounts are displayed, there should be exactly two digits displayed to the right of the decimal point. Formatted numbers such as these can be created as strings in the controller for display in the template. The way to convert a floating-point value to a formatted string is with a variation of the old C language function `sprintf`. This function, which also is named `sprintf`, takes a string parameter that contains a format code, followed by the name of a variable to be converted. The string version is returned by the function. The format codes most commonly used are `f` and `d`. The form of a format

code is a percent sign (%), followed by a field width, followed by the code letter (f or d). The field width for the f code appears in two parts, separated by a decimal point. For example, %f7.2 means a total field width of 7 spaces, with 2 digits to the right of the decimal point, which is perfect for money. The d code field width is just a number of spaces, for example, %5d. So, to convert a floating-point value referenced by the variable @total to a string with two digits to the right of the decimal point, the following could be used:

```
@str = sprintf("%5.2f", @total)
```

The sprintf function is used in the controller for our popcorn application, which is shown in the following program:

```
# home_controller.rb - for the popcorn application
class HomeController < ApplicationController
  def the_form
  end

# result method - fetch data and compute the cost
  def result

# Fetch the form values
    @unpop = params[:unpop].to_i
    @caramel = params[:caramel].to_i
    @caramelnut = params[:caramelnut].to_i
    @toffeynut = params[:toffeynut].to_i
    @name = params[:name]
    @street = params[:street]
    @city = params[:city]
    @payment = params[:payment]

# Compute the item costs and total cost
    @unpop_cost = 3.0 * @unpop
    @caramel_cost = 3.5 * @caramel
    @caramelnut_cost = 4.5 * @caramelnut
    @toffeynut_cost = 5.0 * @toffeynut
    @total_price = @unpop_cost + @caramel_cost +
                   @caramelnut_cost + @toffeynut_cost
    @total_items = @unpop + @caramel + @caramelnut + @toffeynut

# Now convert the dollar amounts to strings with 2 digits
#  to the right of the decimal point
    @total_price = sprintf("%5.2f", @total_price)
    @unpop_cost = sprintf("%5.2f", @unpop_cost)
    @caramel_cost = sprintf("%5.2f", @caramel_cost)
```

```
    @caramelnut_cost = sprintf("%5.2f", @caramelnut_cost)
    @toffeynut_cost = sprintf("%5.2f", @toffeynut_cost)

  end
end
```

The remaining task is to design the view document for the results of an order. This design of this document is based on the corresponding document that was written in PHP in Chapter 11. As there, the customer is presented with a table of the items, both those ordered and those not, along with prices per unit, total of each item ordered, and the cost of each item. Following the table, the total number of items ordered and the total cost of the order are presented. The document `result.rhtml` follows:

```
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- result.rhtml - result view for the popcorn application
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> result.rhtml </title>
  <head>
  <body>

<!-- Display the customer information -->
    <h4> Customer: </h4>
    <%= @name %> <br /> <%= @street %> <br />
    <%= @city %>
    <p /> <p />

<!-- Display a table of the order information -->
    <table border = "border">
      <caption> Order Information </caption>
      <tr>
        <th> Product </th>
        <th> Unit Price </th>
        <th> Quantity </th>
        <th> Item Cost </th>
      </tr>
      <tr align = "center">
        <td> Unpopped Popcorn </td>
        <td> $3.00 </td>
```

```
        <td> <%= @unpop %> </td>
        <td> $<%= @unpop_cost %> </td>
      </tr>
      <tr align = "center">
        <td> Caramel Popcorn </td>
        <td> $3.50 </td>
        <td> <%= @caramel %> </td>
        <td> $<%= @caramel_cost %> </td>
      </tr>
      <tr align = "center">
        <td> Caramel Nut Popcorn </td>
        <td> $4.50 </td>
        <td> <%= @caramelnut %> </td>
        <td> $<%= @caramelnut_cost %> </td>
      </tr>
      <tr align = "center">
        <td> Toffey Nut Popcorn </td>
        <td> $5.00 </td>
        <td> <%= @toffeynut %> </td>
        <td> $<%= @toffeynut_cost %> </td>
      </tr>
    </table>
    <p /><p />

<!-- Display the number of items ordered, the total cost,and
     the chosen method of payment   -->

    <p>
      You ordered <%= @total_items %> popcorn items <br />
      The total cost of your order is $<%= @total_price %> <br />
      Your chosen method of payment is: <%= @payment %> <br />
      Thank you for your order
    </p>
  </body>
</html>
```

Figure 15.7 shows a completed popcorn order form.

Figure 15.8 shows the returned document as it would be displayed on the customer's browser.

Welcome to Millenium Gymnastics Booster Club Popcorn Sales

Buyer's Name: Joe Popcorn

Street Address: 123 Popcorn Lane

City, State, Zip: Popcorn City, Iowa 22222

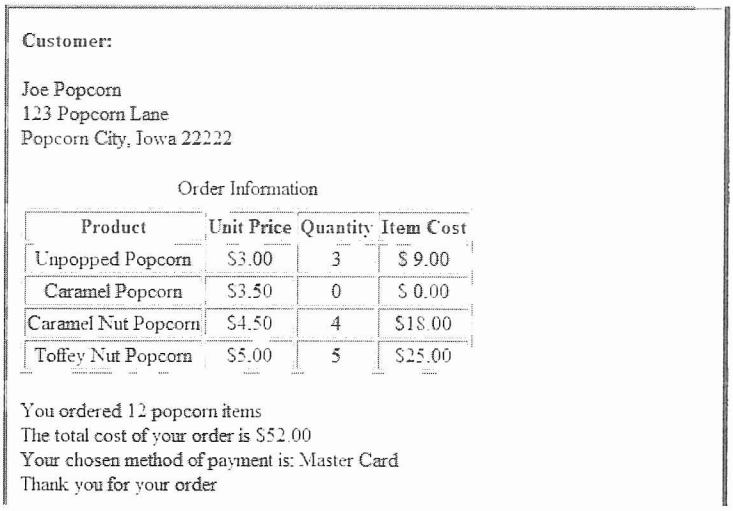
Product Name	Price	Quantity
Unpopped Popcorn (1 lb.)	$3.00	3
Caramel Popcorn (2 lb. cannister)	$3.50	
Caramel Nut Popcorn (2 lb. cannister)	$4.50	4
Toffey Nut Popcorn (2 lb. cannister)	$5.00	5

Payment Method:

- ○ Visa
- ◉ Master Card
- ○ Discover
- ○ Check

Submit Order | Clear Order Form

Figure 15.7 A completed popcorn order form



Customer:

Joe Popcorn
123 Popcorn Lane
Popcorn City, Iowa 22222

Order Information

Product	Unit Price	Quantity	Item Cost
Unpopped Popcorn	$3.00	3	$ 9.00
Caramel Popcorn	$3.50	0	$ 0.00
Caramel Nut Popcorn	$4.50	4	$18.00
Toffey Nut Popcorn	$5.00	5	$25.00

You ordered 12 popcorn items
The total cost of your order is $52.00
Your chosen method of payment is: Master Card
Thank you for your order

Figure 15.8 The `result` template to the customer

15.4 Rails Applications with Databases

This section uses an example application to describe how a Rails application that uses a database is constructed. For the example database, the `cars` database from Chapter 13 is used. The operations that are implemented are simple: The user is presented with a welcome document that states the number of cars listed in the database and the number of states in which they are located. It also presents a form that allows the user to specify the beginning and ending model years in which he or she is interested, as well as a specific body style. The system searches the database for the entries that fit the given restrictions and displays them for the user.

15.4.1 Building the Database

The first step is to build the database. Although the complete database constructed in Chapter 13 could be used, here we describe how to construct such a database and where it must be placed for a Rails application.

As always, we begin by creating the application, which in this case we name `cars`, with the `rails` command. We then move to the new `cars` directory and generate a controller named `main` with the following command:

```
>ruby script/generate controller main
```

So, in this example the controller is named `MainController`. It resides in the file named `main_controller.rb`.

The next step is to construct the database. It is customary in Rails applications to use three copies of the database, one for development, one for testing, and one for production. For this example, however, only a single database will be constructed, `cars_development`. There are several steps involved in creating a database. First, MySQL must be started. This is done with the following command given at the command prompt in the `InstantRails/mysql/bin` directory:

```
>mysql [-h host] [-u username] [database_name] [-p]
```

All of the parameters of this command are optional (as indicated with the brackets). The host defaults to the machine being used, the username defaults to the user who is currently using the machine, and the `-p` flag, when present, indicates that a password must be used to access the database. For this application, the following command to start MySQL is used:

```
>mysql -u root
```

MySQL responds with the `mysql` prompt, at which MySQL commands can be given. At this prompt, we give the command to initialize a database. For the `cars_development` database, the following is used:

```
mysql> create database cars_development;
```

Recall that all MySQL commands must end with a semicolon. MySQL responds to the `create` command with the following:

```
Query OK, 1 row affected (0.03 sec)
```

Now there is a subdirectory for `cars_development` in the `mysql/data` directory.

To actually create the database, two files of MySQL commands are used, one to construct the tables and one to fill them with data. These are placed in the database subdirectory. The first of the files to construct the databases, `build_tables.sql`, follows:

```
Use cars_development;

CREATE TABLE corvettes (
  id INT(11) NOT NULL AUTO_INCREMENT,
  body_style CHAR(12),
  miles FLOAT,
  year INT(4),
  state_id INT(4) NOT NULL,
  PRIMARY KEY (id)
);

CREATE TABLE equipments (
  id INT(11) NOT NULL AUTO_INCREMENT,
  equip CHAR(10),
  PRIMARY KEY (id)
);

CREATE TABLE states (
  id INT(11) NOT NULL AUTO_INCREMENT,
  state CHAR(20),
  PRIMARY KEY (id)
);

CREATE TABLE corvettes_equipments (
  corvettes_id INT(4) NOT NULL,
  equip_id INT(4)
);
```

Notice that the tables are named in plural form. This is required by the Rails system, because the singular form of these names are used for the names of the Ruby files for the models and for the classes associated with the tables.

The `build_tables.sql` file can be executed by MySQL with the following command, given in the `cars_development` directory:

```
>mysql -u root < build_tables.sql
```

If we now go back to the `mysql/bin` directory, start MySQL, and give the `show tables` command, the following is displayed:

```
+--------------------------------+
|                                |
| Tables_in_cars_development     |
|                                |
+--------------------------------+
| corvettes                      |
| corvettes_equipments           |
| equipments                     |
| states                         |
+--------------------------------+
5 rows in set (0.00 sec)
```

The next step is to create the model files for the application. These are generated with the `generate` script using the `model` parameter. For example:

```
>ruby script/generate model corvette
```

This generates a file named `corvette.rb` in the `app/model` directory. This file contains the following class:

```
class Corvette < ActionRecord::Base
end
```

The class is empty, except for the facilities it inherits, which are substantial and essential. They provide methods for setting and getting all of the entity properties of the table, in this case, `body_style`, `miles`, `year`, and `state_id`. All of these are provided implicitly by Rails.

Directives must be added to the model classes to indicate to Rails relationships among the tables of the database. Specifically, there are the *has many* and *belongs to* relationships, which can be specified in the classes associated with the database tables with the `has_many` and `belongs_to` methods. When these relationships are provided, Rails generates the capabilities to manipulate the given relationships.

In the `cars` database, the `corvettes_equipments` table has the primary key, `corvettes_id`, for some rows of the `corvettes` table. Therefore, the `Corvette` class needs the following method call:

```
has_many :corvettes_equipments
```

Likewise, because the `corvettes_equipments` table has the primary key, `equip_id`, for some rows of the `equipments` table, the `Equipment` class needs the following method call:

```
has_many :corvettes-equipments
```

Finally, because the `corvettes` table has the primary key, `state_id`, for some of the rows of the `states` table, the `State` class needs the following method call:

```
has_many :corvettes
```

Rails builds collections for each `has_many` directive. For example, the `has_many :corvettes` directive in the `State` class causes Rails to build a collection of all of the `Corvette` objects for each object of `State`. These are available to the application through the `corvette` method.

Next, we consider the `belongs_to` relationship, which is the opposite of the `has_many` relationship. Therefore, the `CorvettesEquipment` class should have both of the following method calls:

```
belongs_to :corvette
belongs_to :equipment
```

Likewise, the `Corvette` class needs the following method call:

```
belongs_to :state
```

The final forms of the model classes are as follows:

```
class Corvette < ActiveRecord :: Base
  belongs_to :state
end

class Equipment < ActiveRecord::Base
  has_many :corvettes_equipments
end

class State < ActiveRecord::Base
end

class CorvettesEquipment < ActiveRecord::Base
  belongs_to :corvette
  belongs_to :equipment
  has_many :corvettes
end
```

Next, we develop another MySQL file to fill the tables of `cars_development` with some useful data. The file to do this follows:

```
USE cars_development;
INSERT INTO corvettes values
  (1, "coupe", 18.0, 1997, 4),
  (2, "hatchback", 58.0, 1996, 7),
  (3, "convertible", 13.5, 2001, 1),
  (4, "hatchback", 19.0, 1995, 2),
  (5, "hatchback", 25.0, 1991, 5),
```

```
  (6, "hardtop", 15.0, 2000, 2),
  (7, "coupe", 55.0, 1979, 10),
  (8, "convertible", 17.0, 1999, 5),
  (9, "hardtop", 17.0, 2000, 5),
  (10, "hatchback", 50.0, 1995, 7);

INSERT INTO equipments values
  (1, "Automatic"),
  (2, "4-speed"),
  (3, "5-speed"),
  (4, "6-speed"),
  (5, "CD"),
  (6, "Leather");

INSERT INTO states values
  (1, "Alabama"),
  (2, "Alaska"),
  (3, "Arizona"),
  (4, "Arkansas"),
  (5, "California"),
  (6, "Colorado"),
  (7, "Connecticut"),
  (8, "Delaware"),
  (9, "Florida"),
  (10, "Georgia");

INSERT INTO corvettes_equipments values
  (1, 1),
  (1, 5),
  (1, 6),
  (2, 1),
  (2, 5),
  (2, 6),
  (3, 1),
  (3, 6),
  (4, 2),
  (4, 6),
  (5, 1),
  (5, 6),
  (6, 2),
  (7, 4),
  (7, 5),
  (8, 4),
  (8, 5),
  (8, 6),
```

```
(9, 4),
(9, 5),
(9, 6),
(10, 1),
(10, 5);
```

This file is executed with the following command:

```
>mysql -u root < fill_tables.sql
```

15.4.2 Building the Application

Now we have a database against which we can develop and test our application. The next step is to write the first action method of our controller. This method will provide the data required for the first template of the application, which we named `welcome`. This template welcomes the user and indicates the number of cars listed and the number of states in which they are located. Therefore, the `welcome` method must provide those two numbers. The number of rows in a table can be determined by calling the `count` method on the table's object. For example, the number of rows in the `corvettes` table is gotten with `Corvette.count`.

Every model class (therefore, every database table) supports the `find` method, which searches its table for rows that satisfy given criteria. The simplest way to use `find` is to pass it one or more primary keys. For example:

```
mycar = Corvettes.find(8)
```

If given more than one key, `find` returns an array of the requested row objects. For example:

```
list_five = (1, 2, 3, 4, 5)
first_five = Corvettes.find(list_five)
```

A `RecordNotFound` exception is thrown if any of the requested primary keys cannot be found.

In most cases, row objects of tables are needed that meet certain criteria. The `find` method can do this, too. If the first parameter to `find` is `:all`, `find` searches can be controlled by a second parameter, which is specified as the value of the `:conditions` symbol. For example, consider the following statement:

```
sixty_five = find(:all, :conditions => "year = 1965")
```

More than one condition can be specified, as shown in the following:

```
sixty_five_conv = find(:all, :conditions => "year = 1965
                              and body_style = 'convertible'")
```

This form of call to `find` is adequate only if the conditions are all literals. In many cases, the condition is at least partially made up of user input, often

form data. For example, if the `year` condition value were in the `@year` instance variable, the following could be used:

```
sixty_five_conv = find(:all, :conditions =>
          "year = #{@year} and body_style = 'convertible'")
```

This approach has a critical drawback—it allows a security problem named *SQL injection attack*. So, Rails has a safe alternative, which is to use a different form of `:conditions` value. With this form, the value of `:conditions` is delimited with brackets, question marks appear in place of the user-input values, and the condition is followed by a comma and the variables that have the values. The new form of the example above is as follows:

```
sixty_five_conv = find(:all, :conditions =>
      ["year = ? and body_style = 'convertible'", @year])
```

If the first parameter to `find` is `:first`, it returns the first row object it finds that meets the specified condition.

If the user is comfortable with SQL, the `find_by_sql` method might be preferable. It takes an SQL `SELECT` command, in literal string form, as a parameter and returns a collection of row objects.

Following is the controller with the `welcome` method for the `cars` application, which only uses simple forms of `find`:

```
# main_controller.rb - for the cars application
class MainController < ApplicationController

# welcome method - fetches values for the
#  initial view

  def welcome
    @num_cars = Corvette.count
    @num_states = State.count
  end
end
```

Now we can develop the `welcome` template, which is stored in the `welcome.rhtml` file. This document must give the initial information and then display a form that the user can fill in and submit to learn about specific cars for sale. The `welcome` document uses the two values produced by the `welcome` method of the controller, `@num_cars` and `@num_states`. The `welcome.rhtml` file follows:

```
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- welcome.rhtml - initial view for the cars application
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> welcome template for cars </title>
  </head>
  <body>

<!-- The initial information -->
    <p>
      <h1> Aidan's Used Car Lot </h1>
      <h2> Welcome to our home document </h2>
      We currently have <%= @num_cars %> used Corvettes listed <br />
      The cars are in <%= @num_states %> different states <br /> <br />
      To request information on available cars, please fill out <br />
      the following form and submit it
    </p>

<!-- The form to collect input from the user about their interests -->

    <form action = "result"  method = "post" >
      From year: <input type = "text"  size = "4"  name = "year1" />
      To year: <input type = "text"  size = "4"  name = "year2" />
      Body style: <input type = "text"  size = "12"  name = "body" />
 <br />
      <input  type = "submit"  value = "Submit request" /> <br />
      <input type = "reset"  value = "Reset form" /> <br />
    </form>
    <hr/>
    <p> Copyright 2007, AUCL, Inc. </p>
  </body>
</html>
```

The display of the `welcome` template is shown in Figure 15.9.

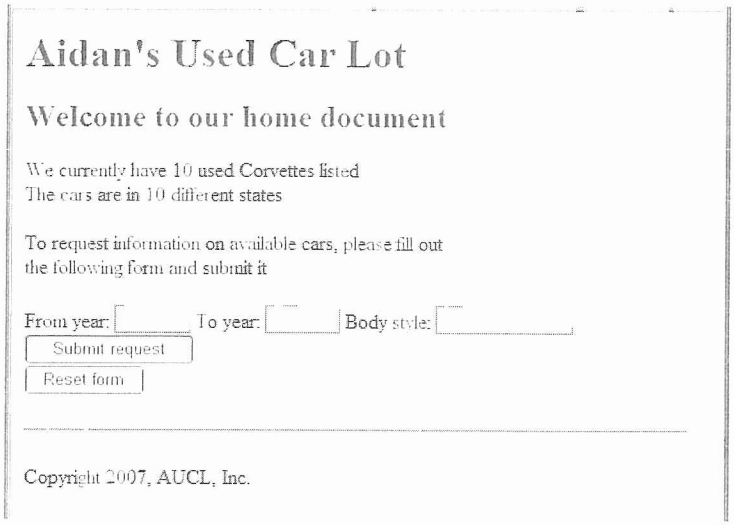


Figure 15.9 Display of `welcome.rhtml`

The next step is to write the result method, which fetches the data from the form, as well as the other required data. The form data are the two years and the body style. The other required data are the rows of the `corvettes` table and the rows of the `states` table. We have already discussed how all of these values are fetched. Following is the complete `MainController` class:

```
# main_controller.rb - for the cars application
class MainController < ApplicationController

# welcome method - fetches values for the
#  initial view

  def welcome
    @num_cars = Corvette.count
    @num_states = State.count
  end

# result method - fetches values for the
#  result view
  def result
    @year1 = params[:year1]
    @year2 = params[:year2]
    @body = params[:body]
    @carz = Corvette.find(:all, :conditions =>
```

```
                    ["year >= ? and year <= ? and body_style = ?",
                       @year1, @year2, @body])
    @statez = State.find(:all)
  end
end
```

In the `MainController` class, we named the collection of cars and states `carz` and `statez` with `z`'s so they will not be confused with the name of the application (`cars`) and the name of the table of the database, `states`, respectively.

The last step of the development of the application is to design the `result` template, which is stored in the `result.rhtml` file. To provide a pleasant appearance, the information about the specified cars is placed in a table. An `each` iterator is used to go through all of the cars in the `@carz` array provided by the `result` method in the controller. An `if` construct is used to determine whether to include a given car in the result.

Recall that the `corvettes` table has the states where the cars are located represented as the `state_id`, essentially a pointer to the `states` table. The actual name of the state can be gotten from the `@statez` array, provided by the `result` method. Because the `state_id` value is an integer, it can be used as a subscript into the `@statez` array. Once the row (object) is gotten, it can be modified by the `state` field name to get the state's name. The following two statements perform these operations, assuming the car's row can be referenced with `car`:

```
@state_row = @statez[car.state_id.to_i]
@state = @state_row.state
```

The complete `result` template file follows:

```
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- result.rhtml - the result of the user request for
                    information about cars
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> result.rhtml </title>
  </head>
  <body>
    <p>
```

```
<!-- Display what the user asked for -->
      Cars from <%= @year1 %> to <%= @year2 %>
       with the <%= @body %> body style
    </p>

<!-- Display the results of the request in a table -->
    <table border = "border">
      <tr>
        <th> Body Style </th>
        <th> Miles </th>
        <th> Year </th>
        <th> State </th>
      </tr>

<!-- Put the cars in @carz in the table -->
      <% @carz.each do |car|
        @year = car.year
        @body_style = car.body_style
        @miles = car.miles
        @state_row = @statez[car.state_id]
        @state = @state_row.state %>
        <tr>
          <td> <%= @body_style %> </td>
          <td> <%= @miles %> </td>
          <td> <%= @year %> </td>
          <td> <%= @state %> </td>
        </tr>
      <% end %> <!-- end of do loop -->
    </table>
  </body>
</html>
```

Finally, the use of the `cars` application can be illustrated. Figure 15.10 shows a display of the `welcome` template, after it has been filled in by a user.

Figure 15.11 shows the `result` template, after the `welcome` form shown in Figure 15.10 has been submitted.

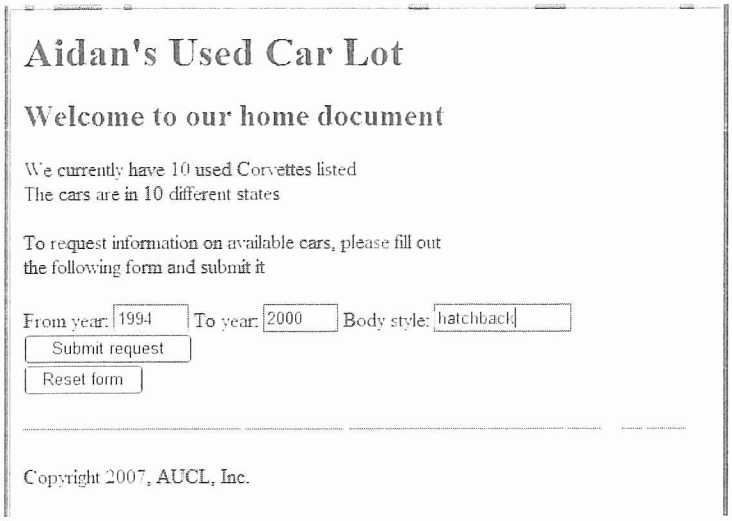


Figure 15.10 A filled–in `welcome` template for `cars`

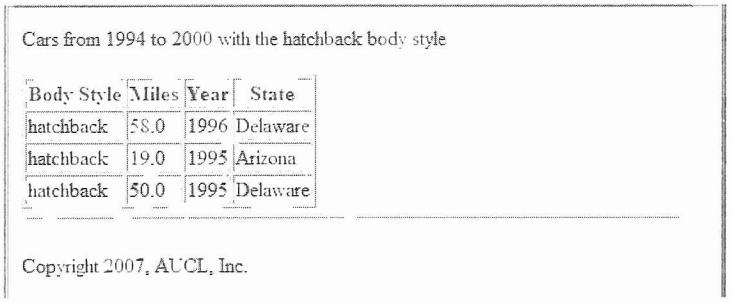


Figure 15.11 The `result` template for `cars`

15.5 Layouts

The `views` directory of each application has two subdirectories, one that has the name of the controller and another subdirectory named `layouts`. Rails creates the `layouts` subdirectory, but leaves it empty. The user can create a "layout" template and place it in the `layouts` directory. A "layout" template is a template for other templates. It provides a way to put some boilerplate markup into each template file in the application. For example, the `cars` application in Section 15.4 has two template files. These two have some common markup.